Arithmetic for Computers

Instructor: Jeongseob Ahn (안정섭)

E-mail: jsahn@csl.korea.ac.kr
Hompage: https://jeongseob.github.io

Introduction

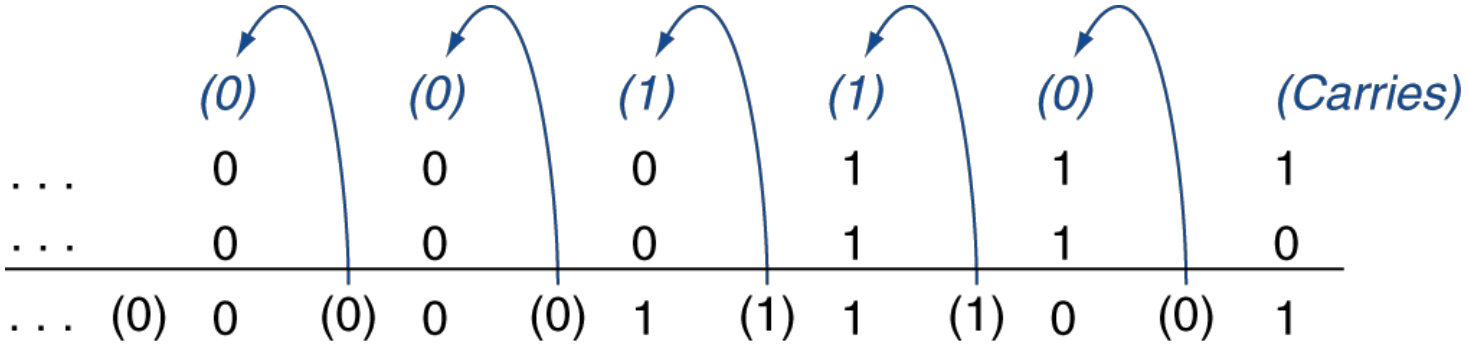
- Bits are just bits
  - No inherent meaning: conventions define relationship between bits and numbers
  - n-bit binary numbers: 0 ~ 2^n-1 decimal numbers
- Of course it gets more complicated
  - Numbers are finite (overflow)
  - Negative numbers
  - Fractions and real numbers
  - Precision and accuracy
  - Error propagation, …

Arithmetic for Computers

- Operations on integers
  - Addition and subtraction
  - Multiplication and division
  - Dealing with overflow
- Floating-point real numbers
  - Representation and operations

Integer Addition

- Example: 7 + 6



- Overflow if result out of range
  - Adding **+ve** and **–ve** operands, no overflow
  - Adding two **+ve** operands
    - Overflow if result sign is 1
  - Adding two **–ve** operands
    - Overflow if result sign is 0

Integer Subtraction

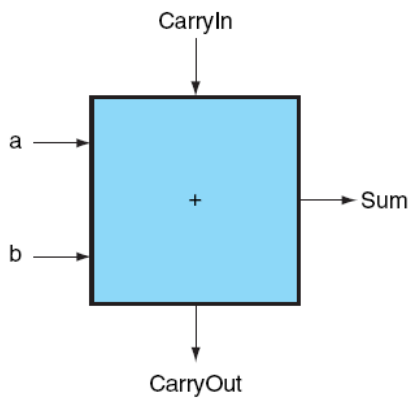
- Add negation of second operand
- Example: 7 - 6 = 7 + (-6)

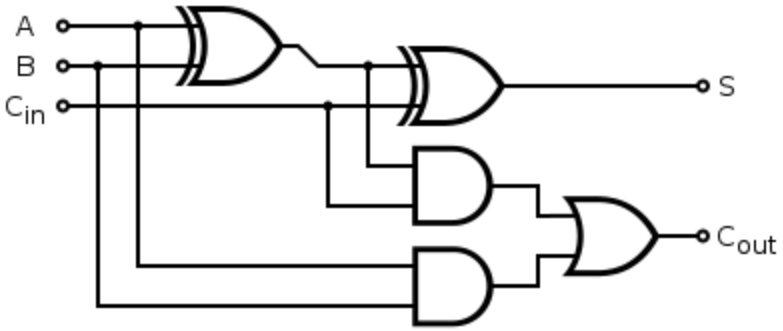
+7:	0000 0000 ⋯ 0000 0111
-6:	1111 1111 ⋯ 1111 1010
+1:	0000 0000 ⋯ 0000 0001

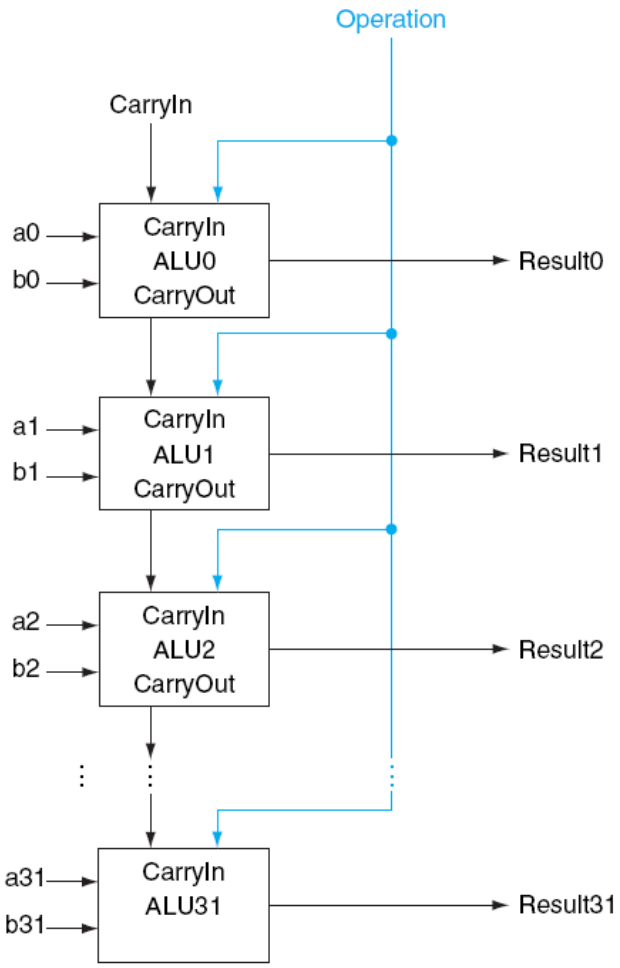
- Overflow if result out of range
  - Subtracting two **+ve** or two **–ve** operands, no overflow
  - Subtracting **+ve** from **–ve** operand
    - Overflow if result sign is 0
  - Subtracting **–ve** from **+ve** operand
    - Overflow if result sign is 1

Simple Adder

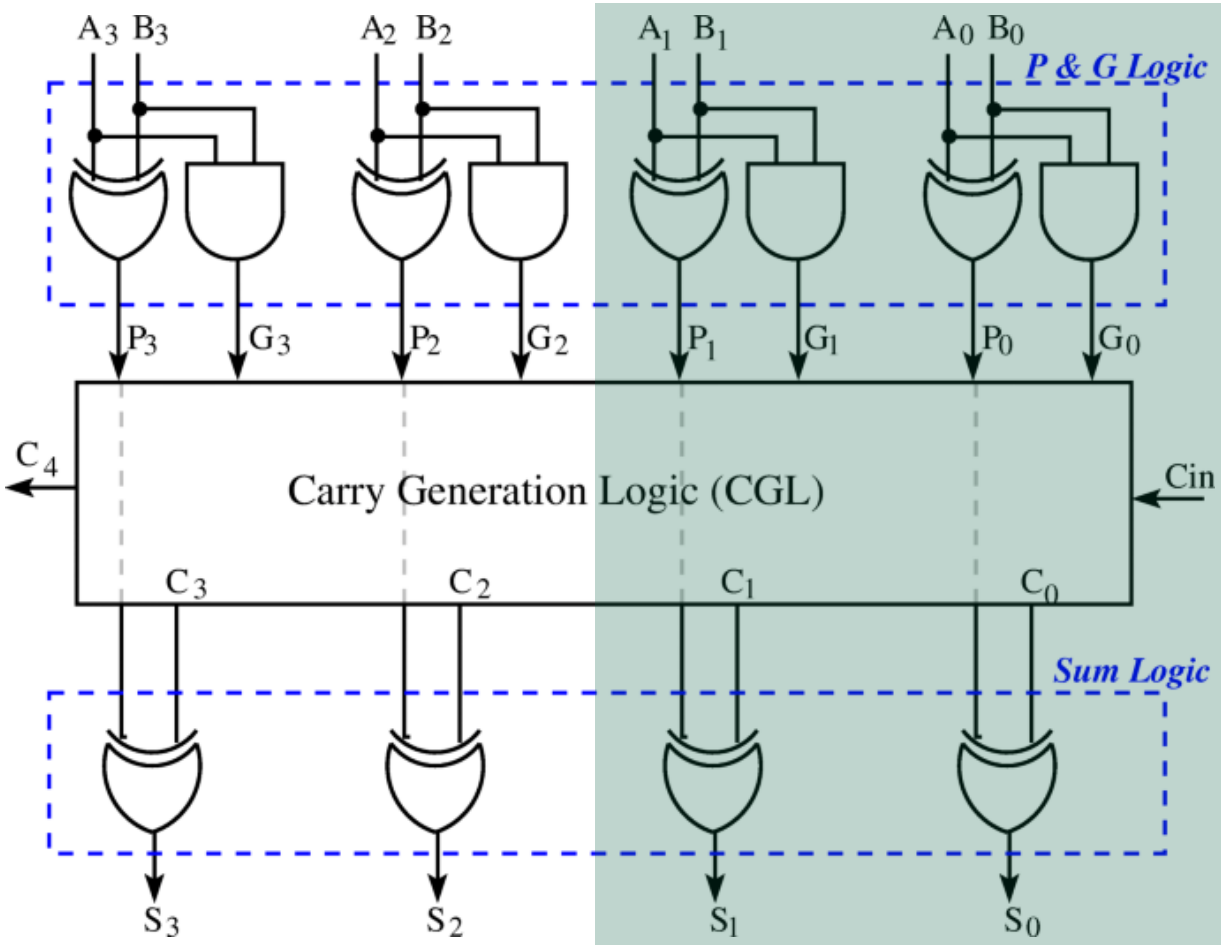
- n-bit ripple-carry adder

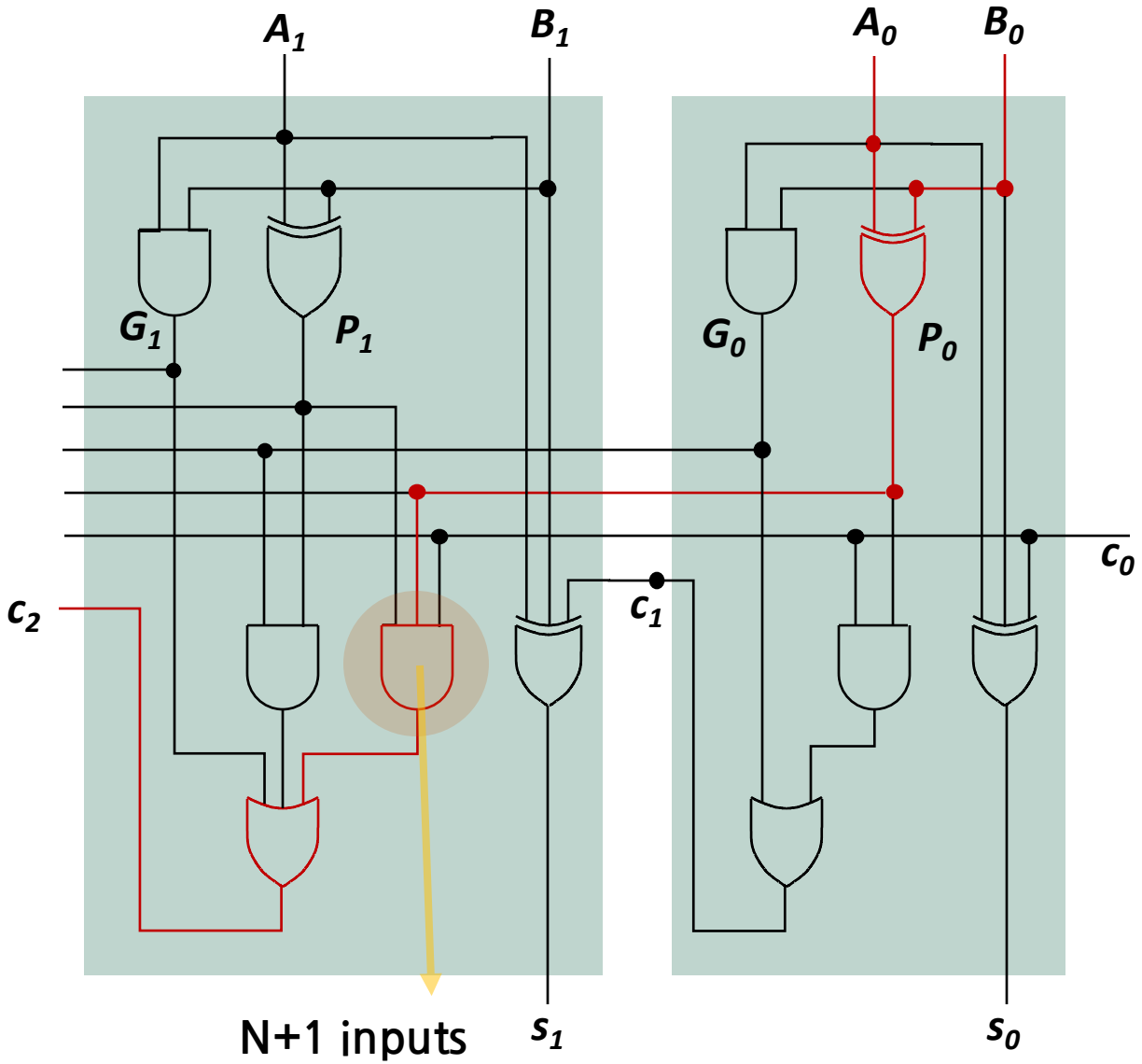






Carry Lookahead Adder





Multiplication

- Start with long-multiplication approach

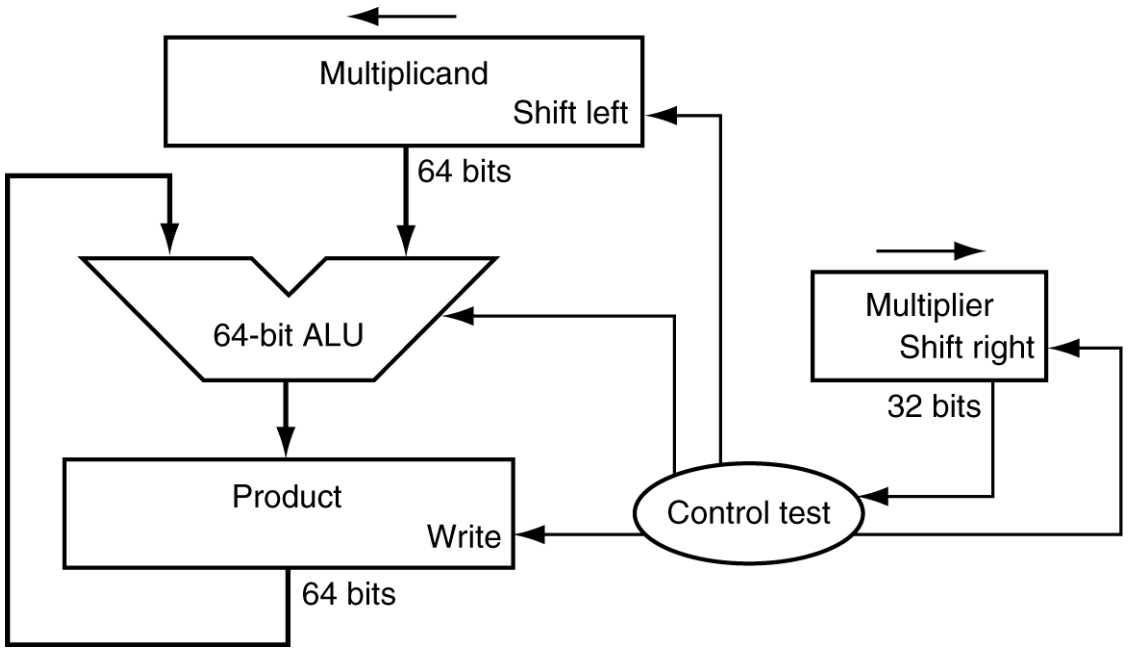
multiplicand

multiplier

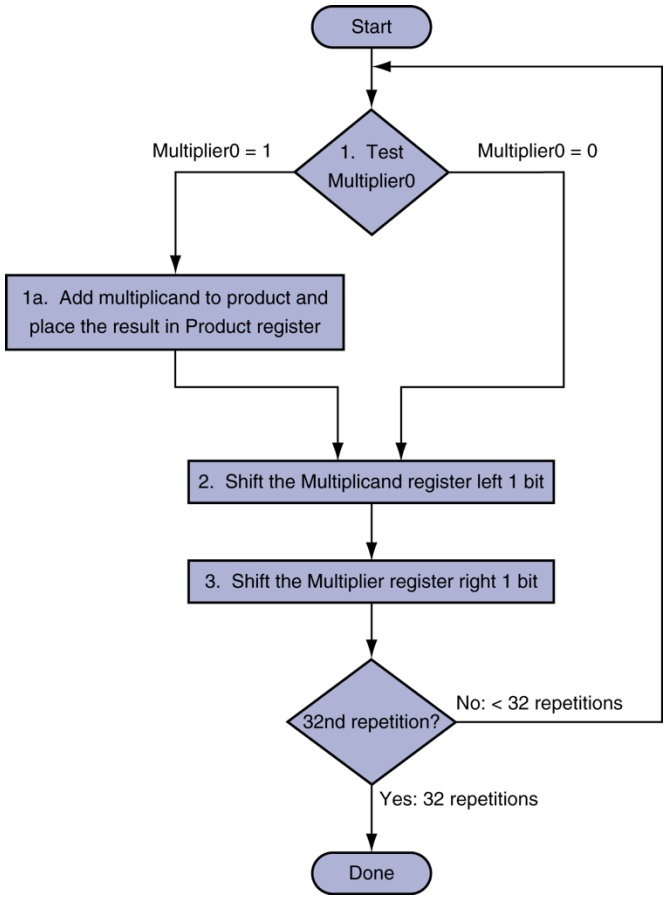
product

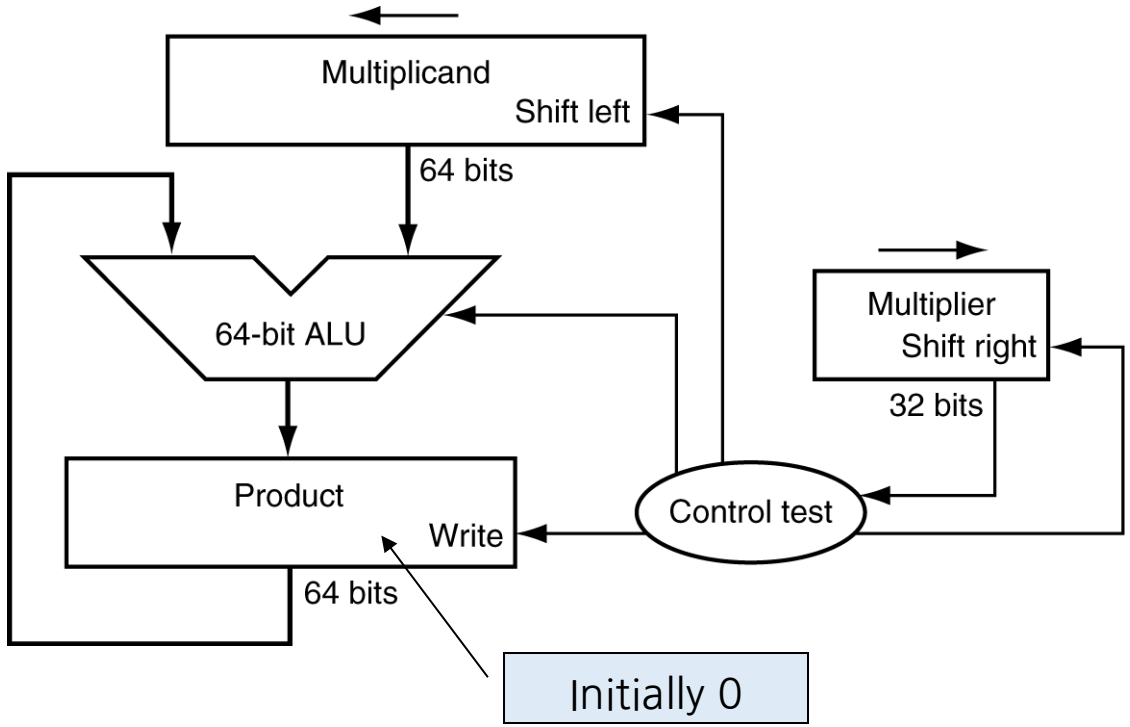
```
      1000
  ×   1001
  --------
      1000
     0000
    0000
   1000
  --------
   1001000
```

Length of product is the sum of operand lengths



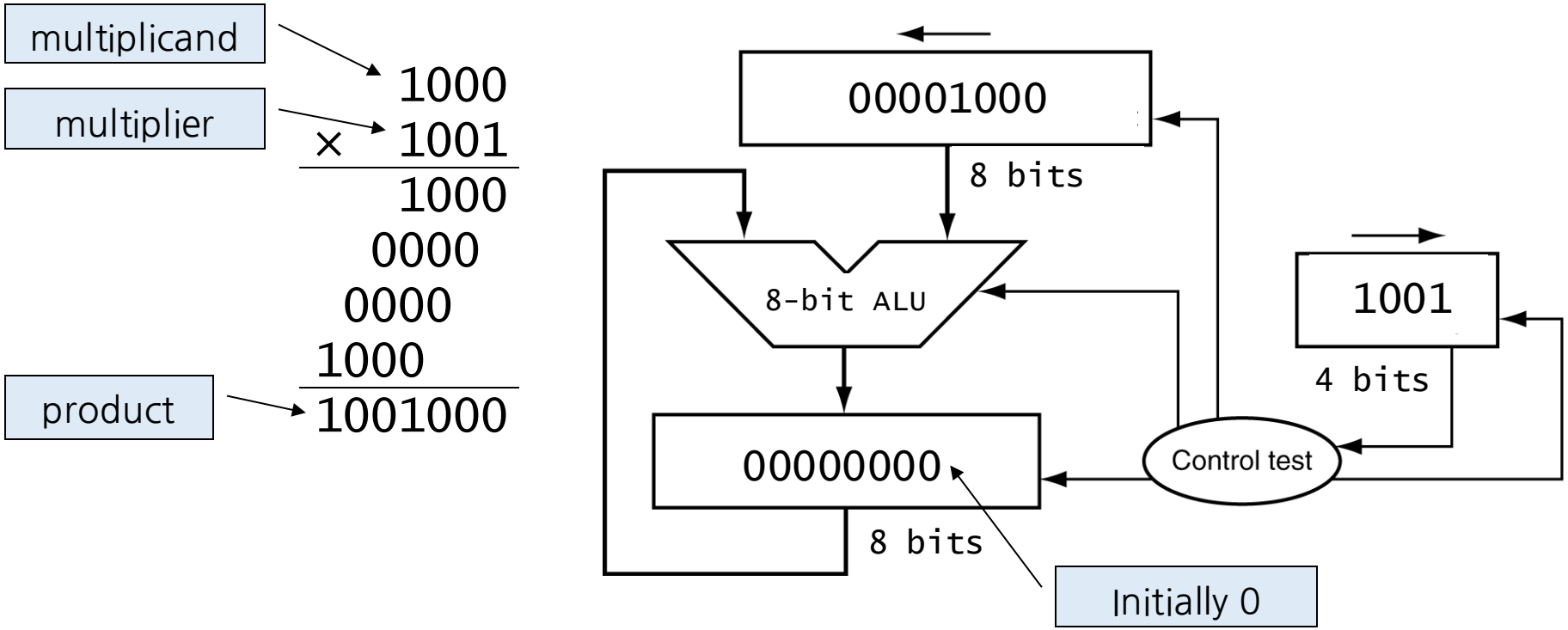
Multiplication Hardware





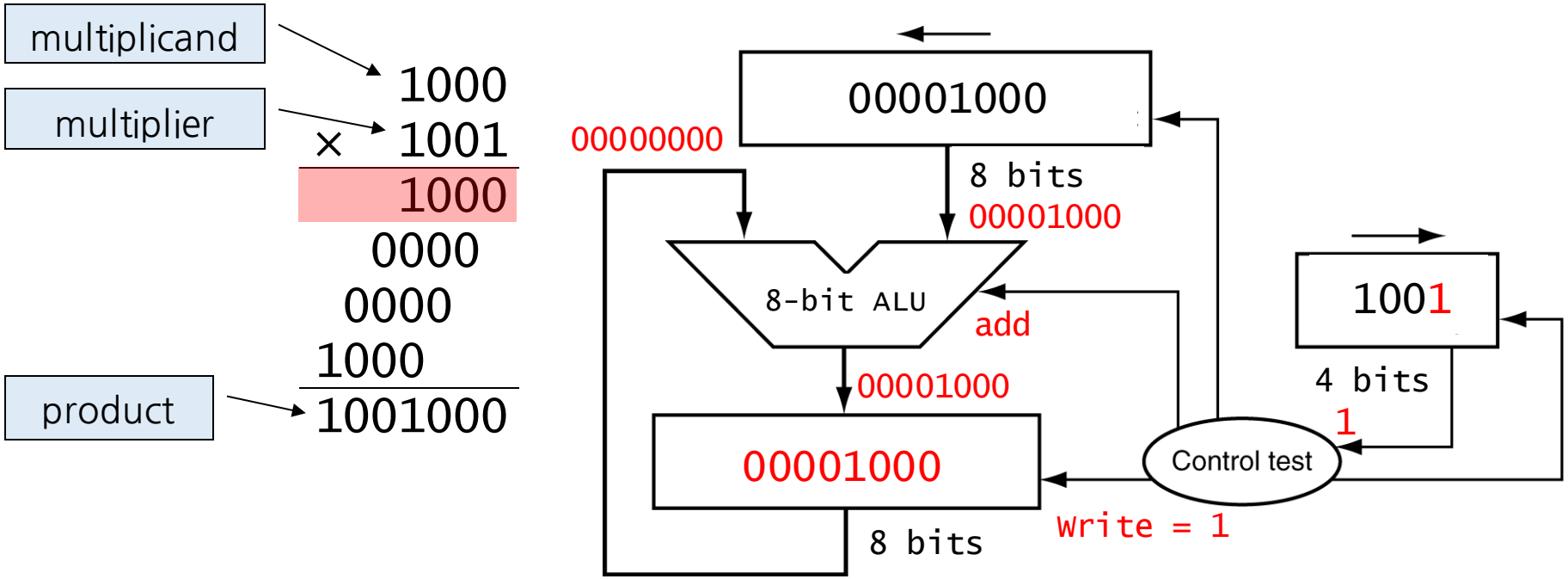
Multiplication Hardware: Example

- Initially



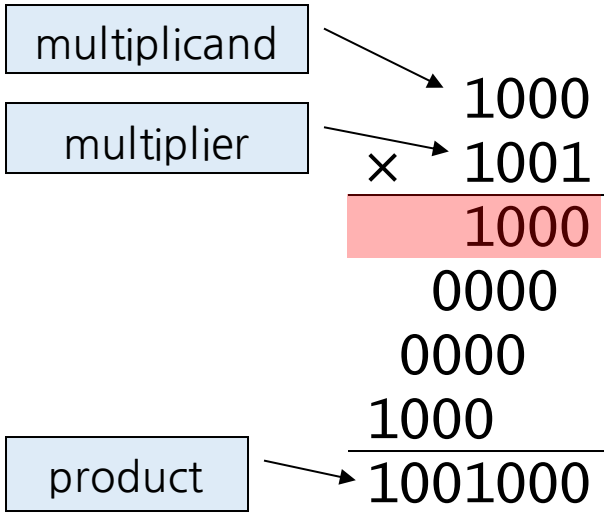
Multiplication Hardware: Example

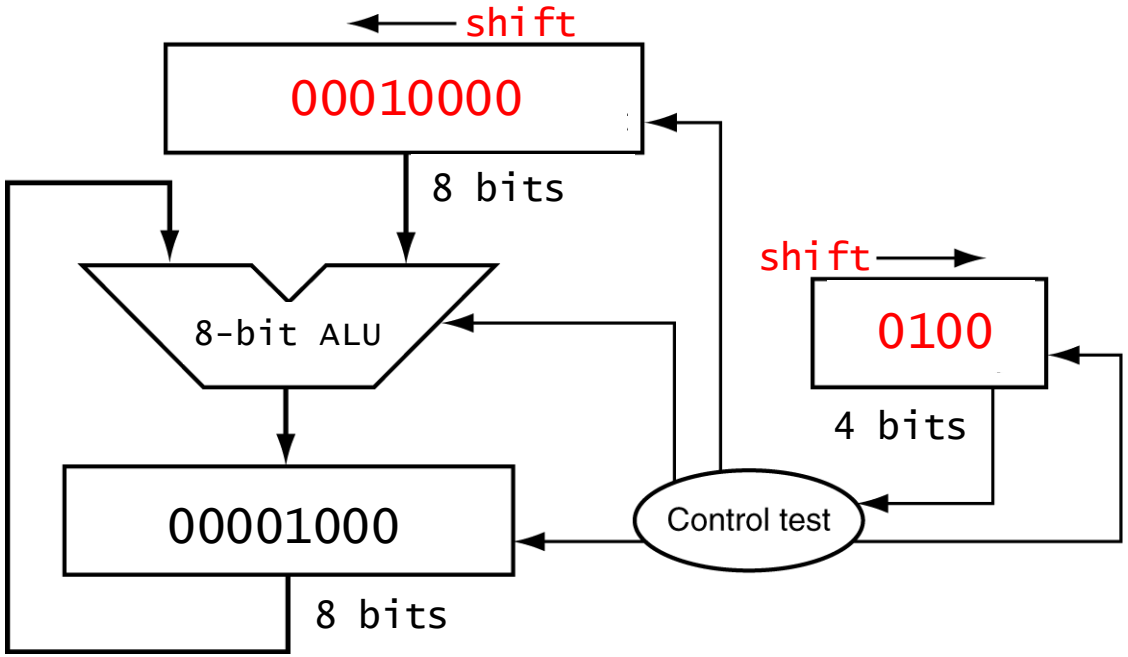
- Iteration 1: after addition



Multiplication Hardware: Example

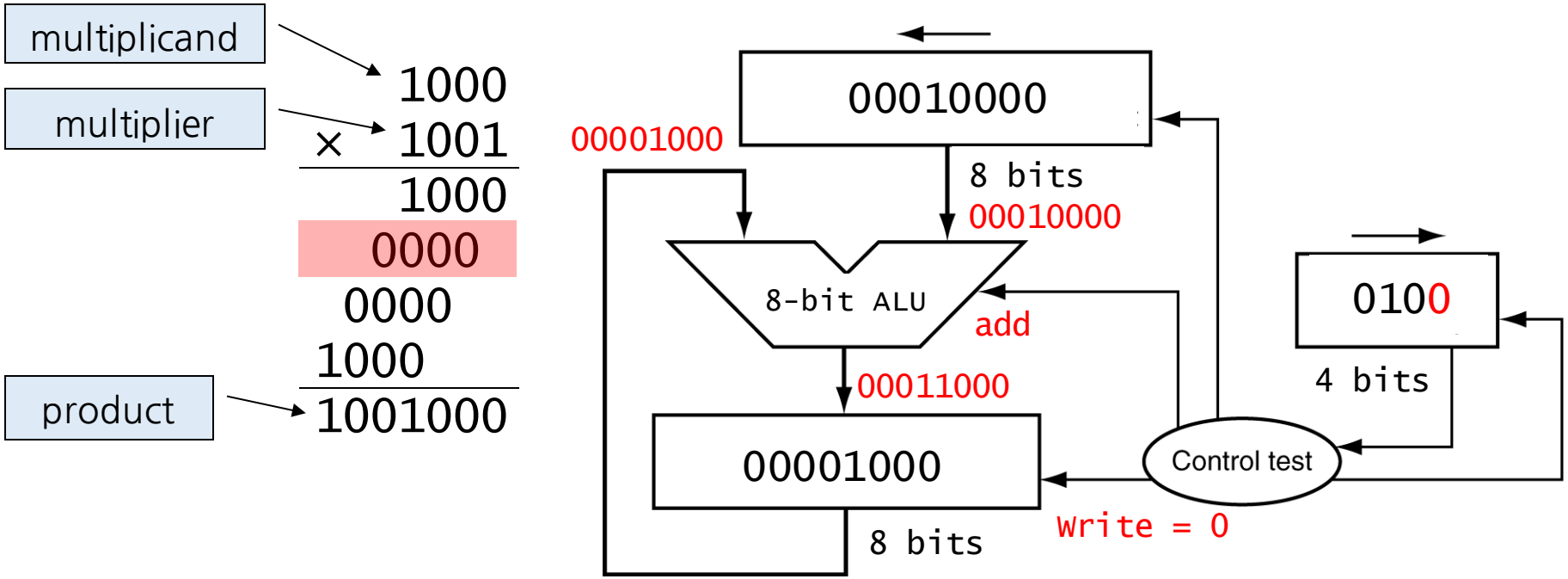
- Iteration 1: after shift





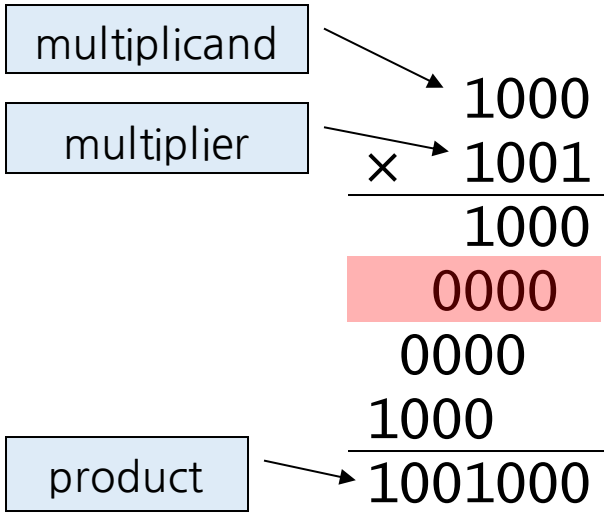
Multiplication Hardware: Example

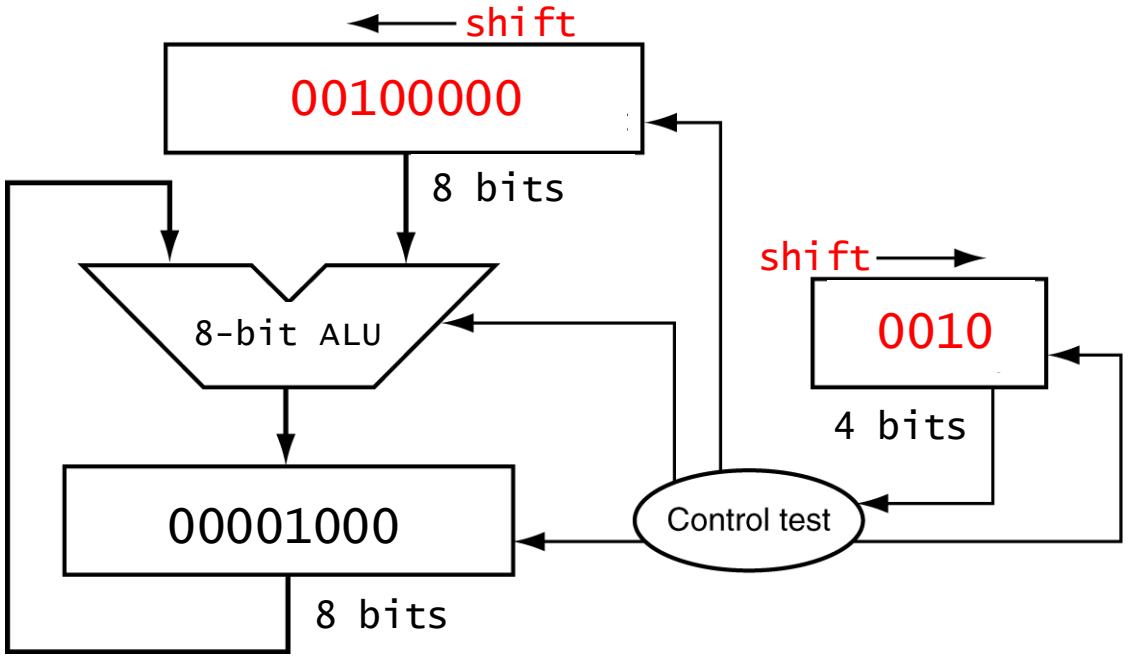
- Iteration 2: begins



Multiplication Hardware: Example

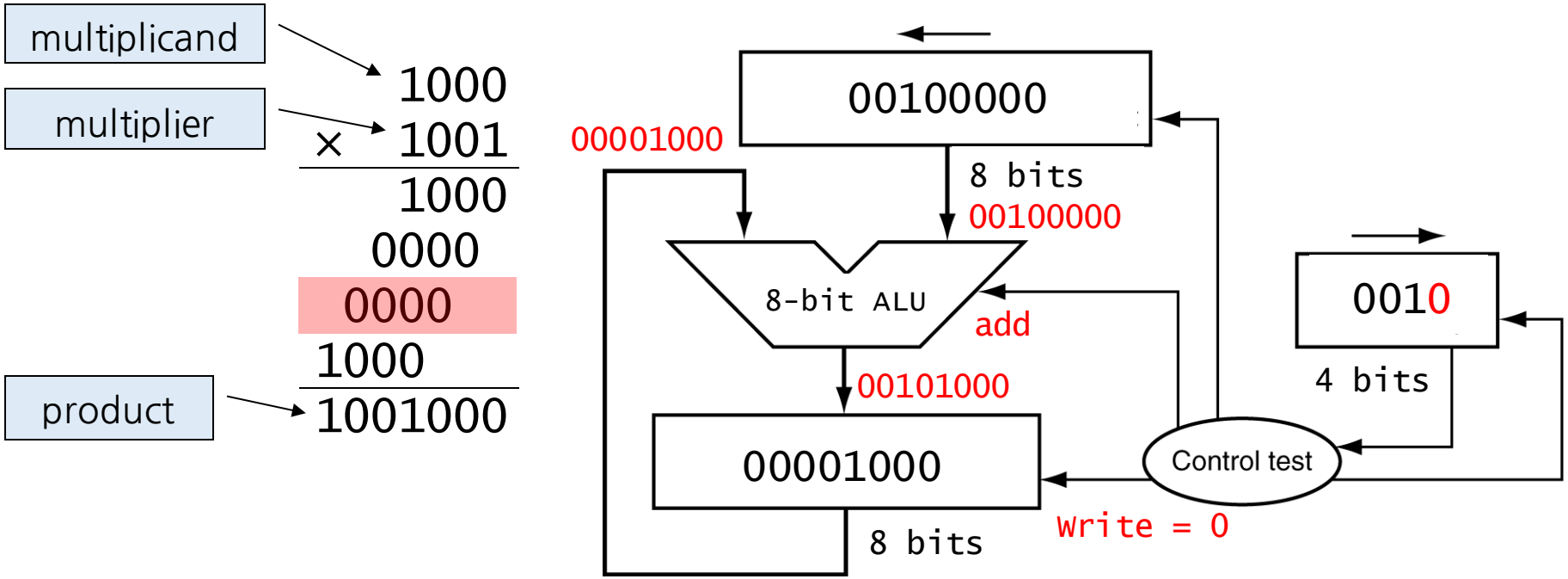
- Iteration 2: after shift





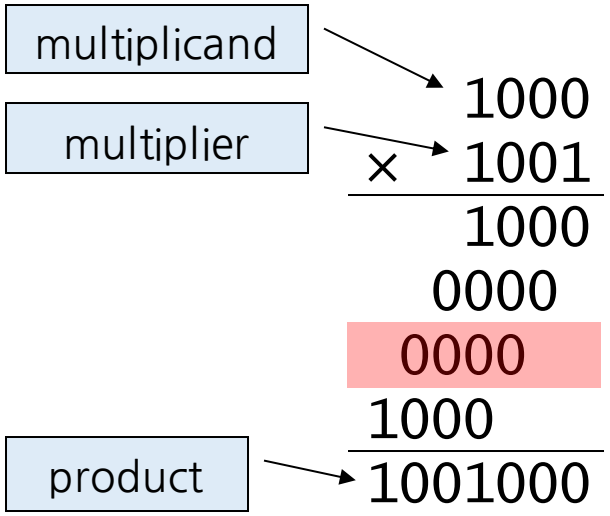
Multiplication Hardware: Example

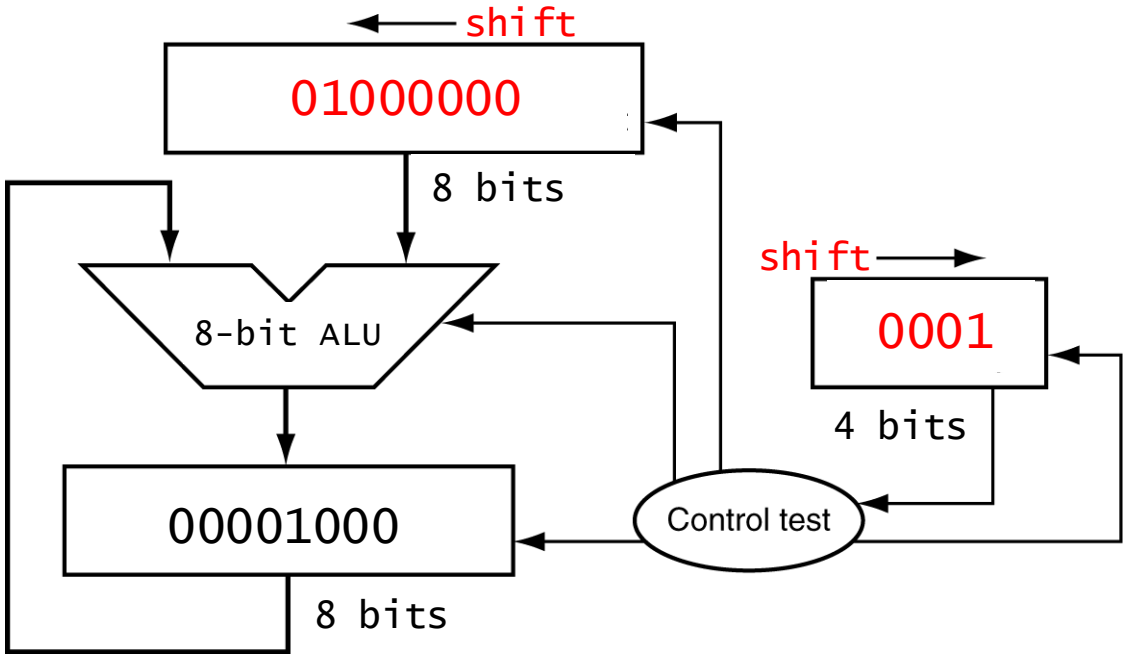
- Iteration 3: begins



Multiplication Hardware: Example

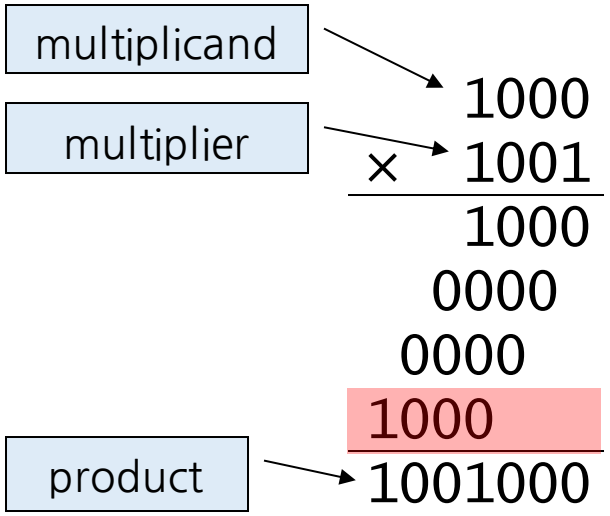
- Iteration 3: after shift

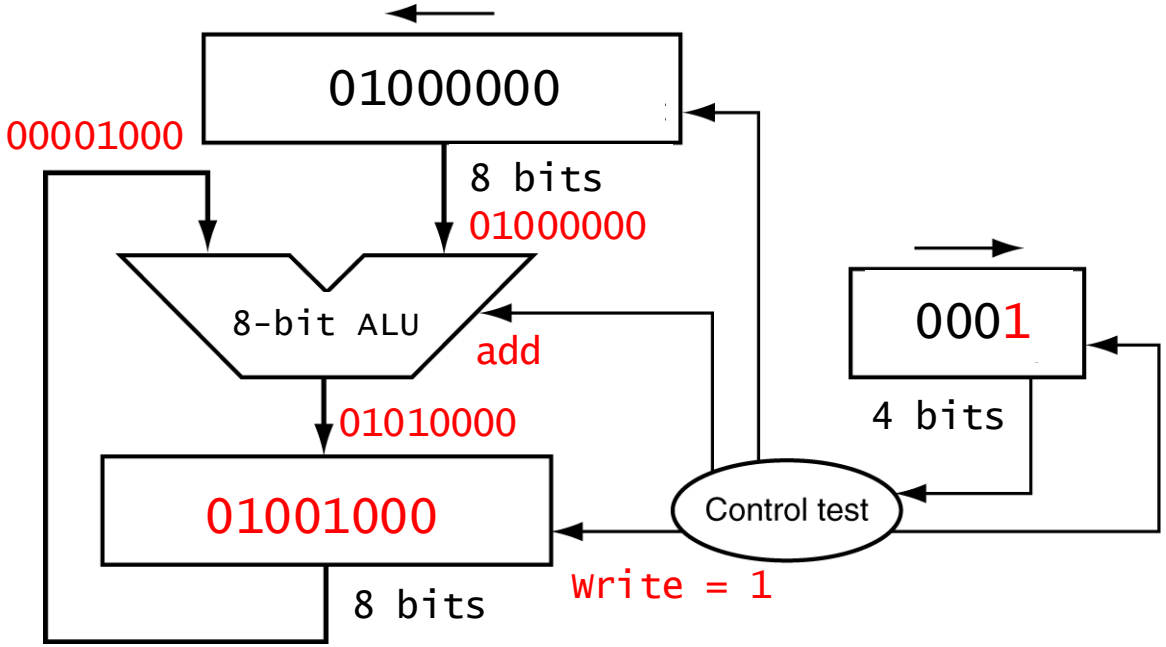




Multiplication Hardware: Example

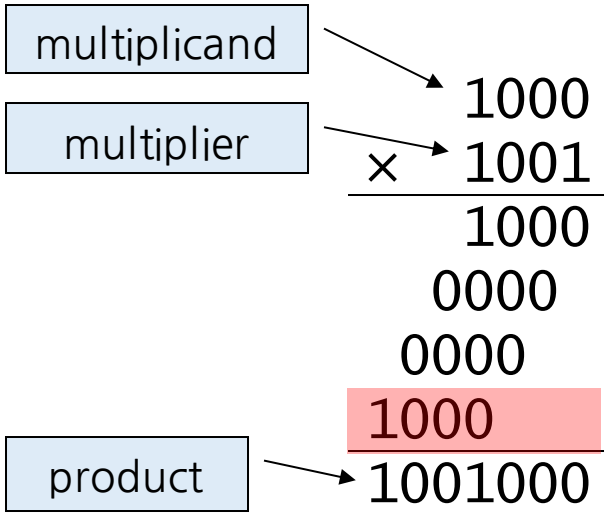
- Iteration 4: begins

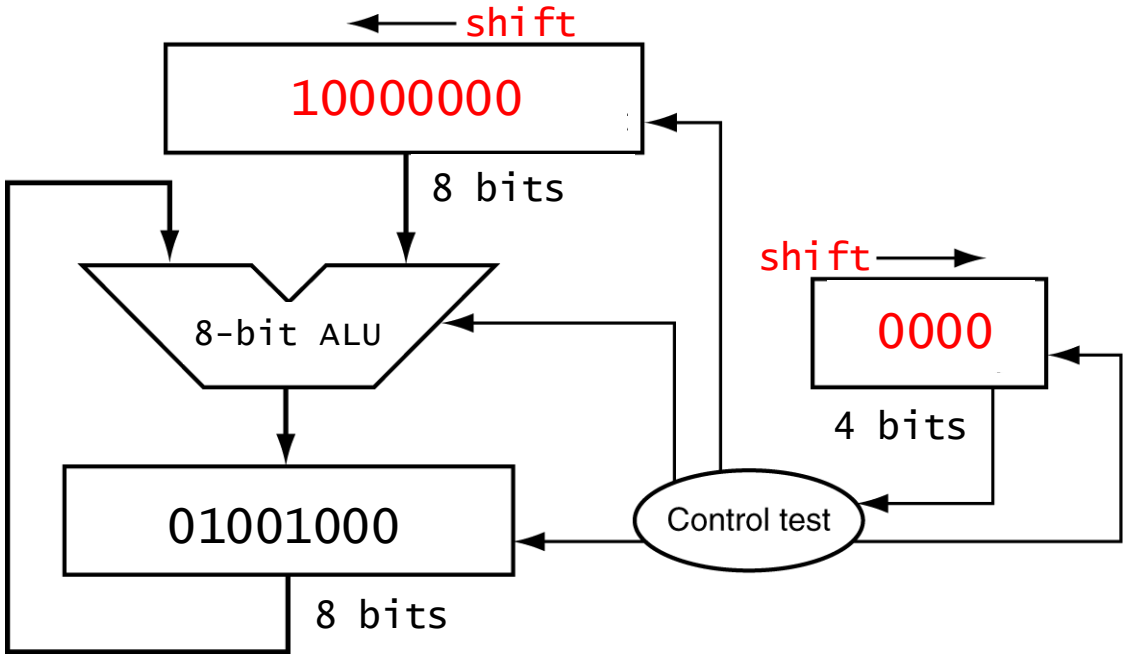




Multiplication Hardware: Example

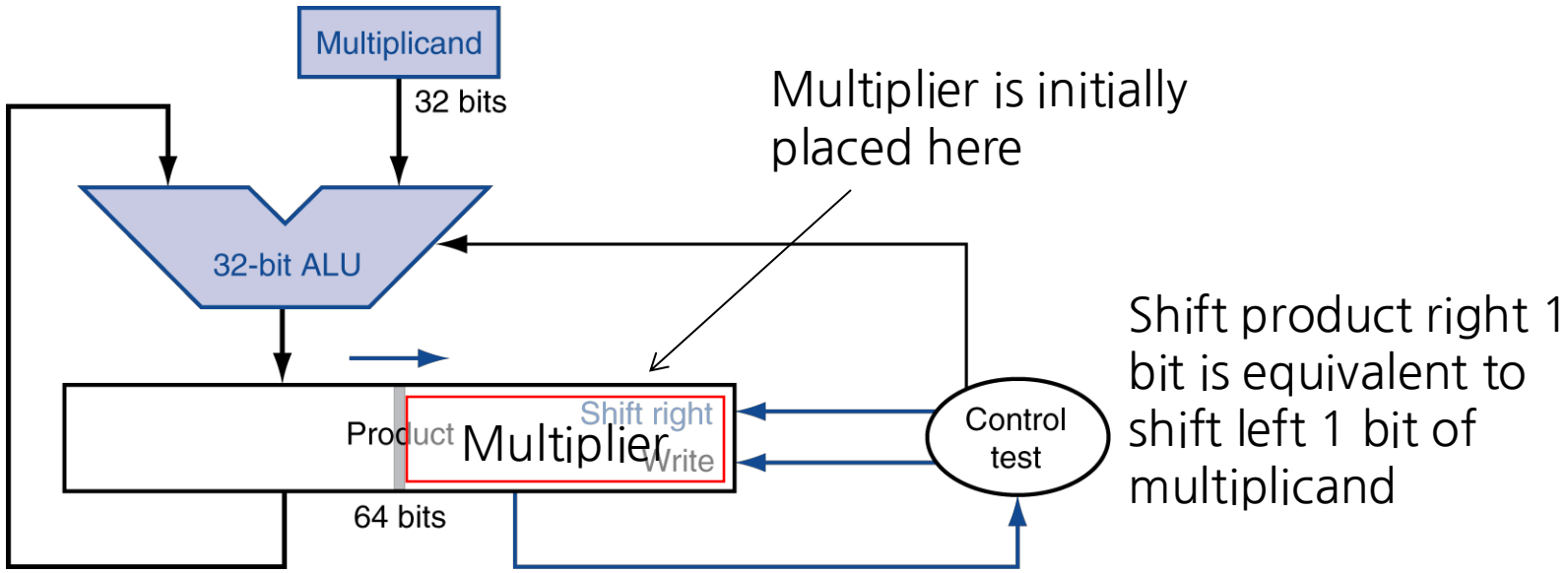
- Iteration 4: after shift





Optimized Multiplier

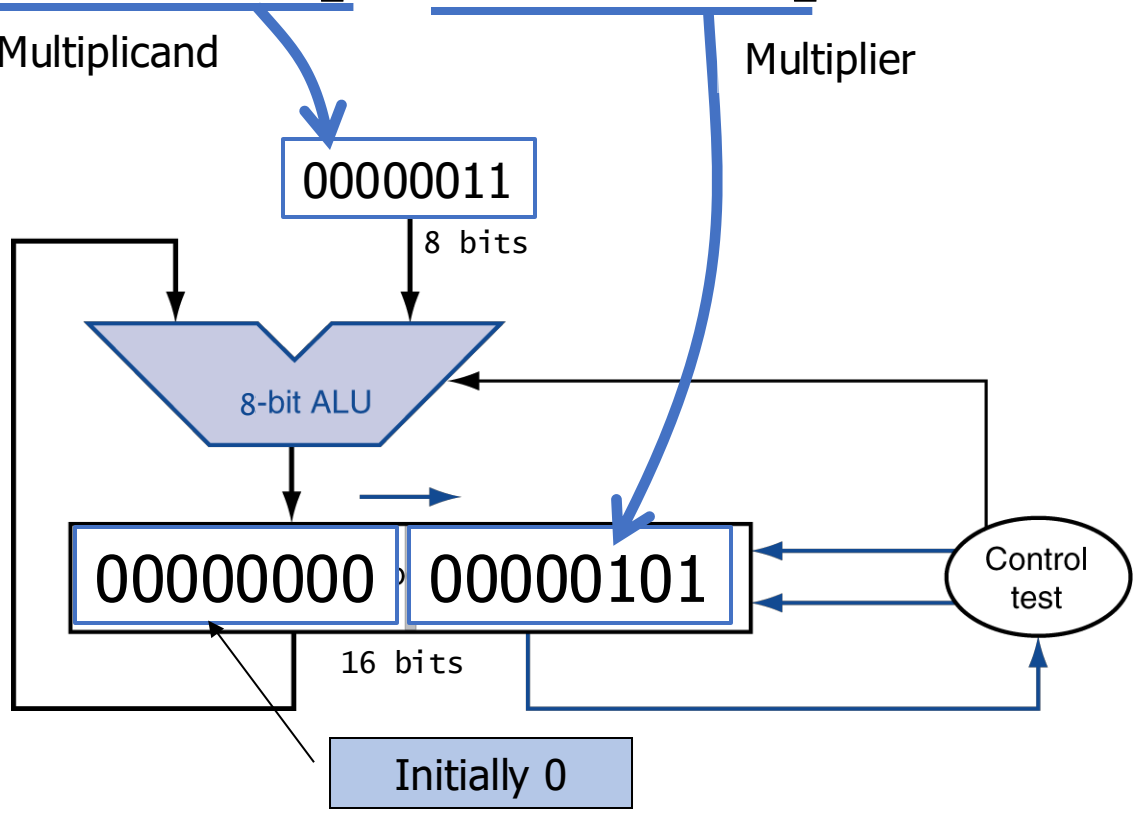
- Perform steps in parallel: add/shift



- One cycle per partial-product addition
  - That's ok, if frequency of multiplications is low

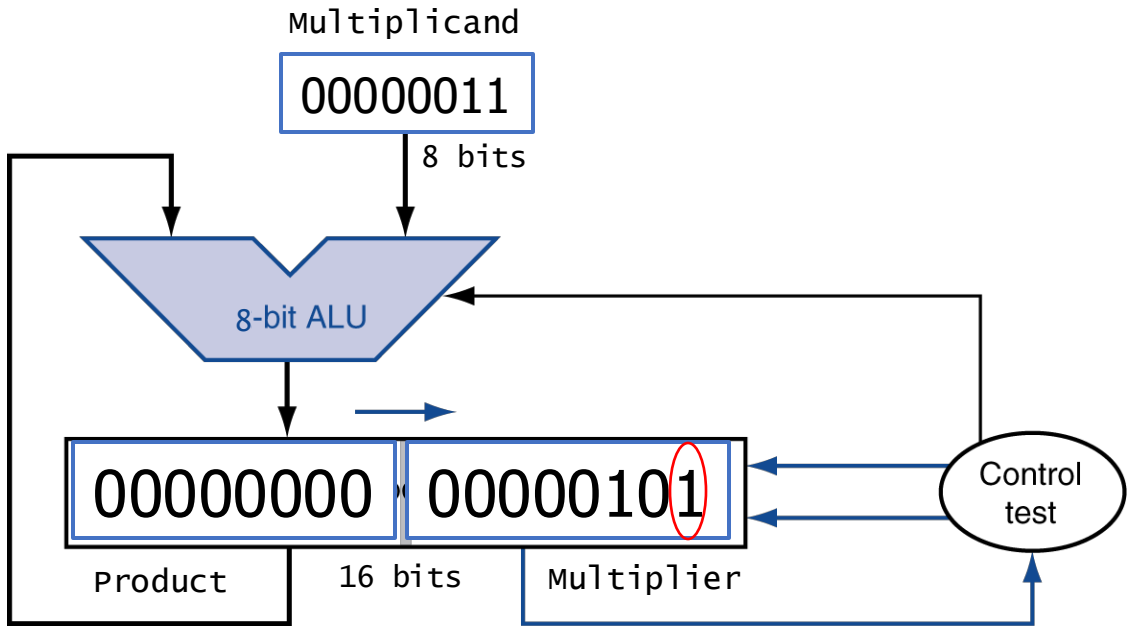
Multiplication Example

- $3_{10} \times 5_{10} = \underline{00000011_2} \times \underline{00000101_2}$



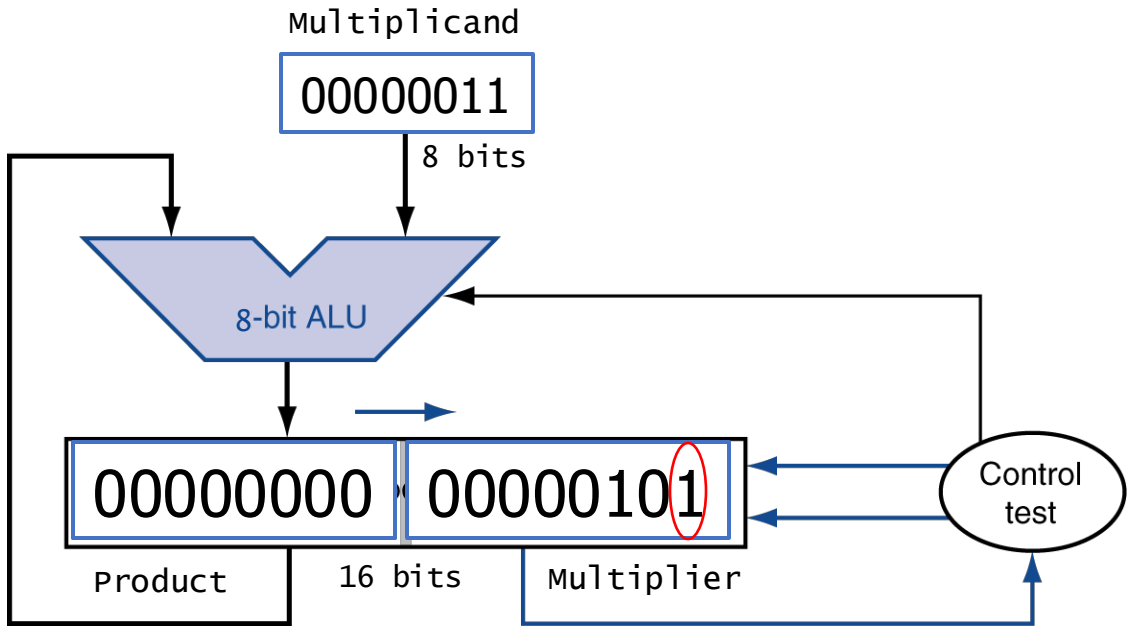
Multiplication Example

- If $\text{Multiplier}_0 = 1$
  - add multiplicand to the left half of product & place the result in the left half of Product register



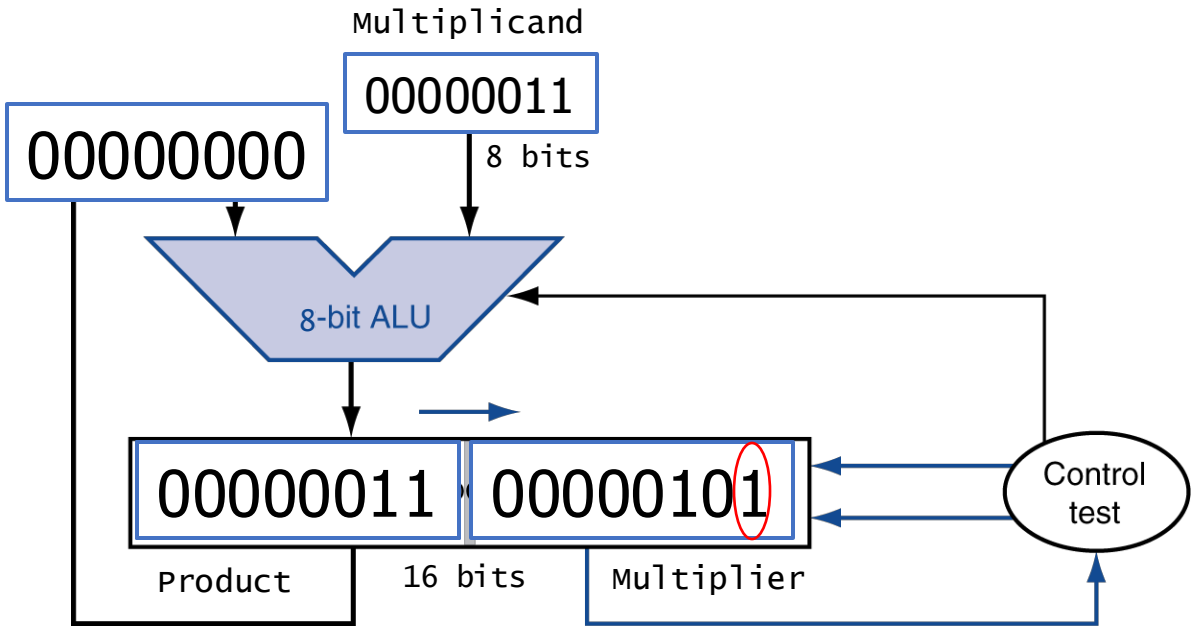
Multiplication Example

- If $\text{Multiplier}_0 = 1$
  - add multiplicand to the left half of product & place the result in the left half of Product register



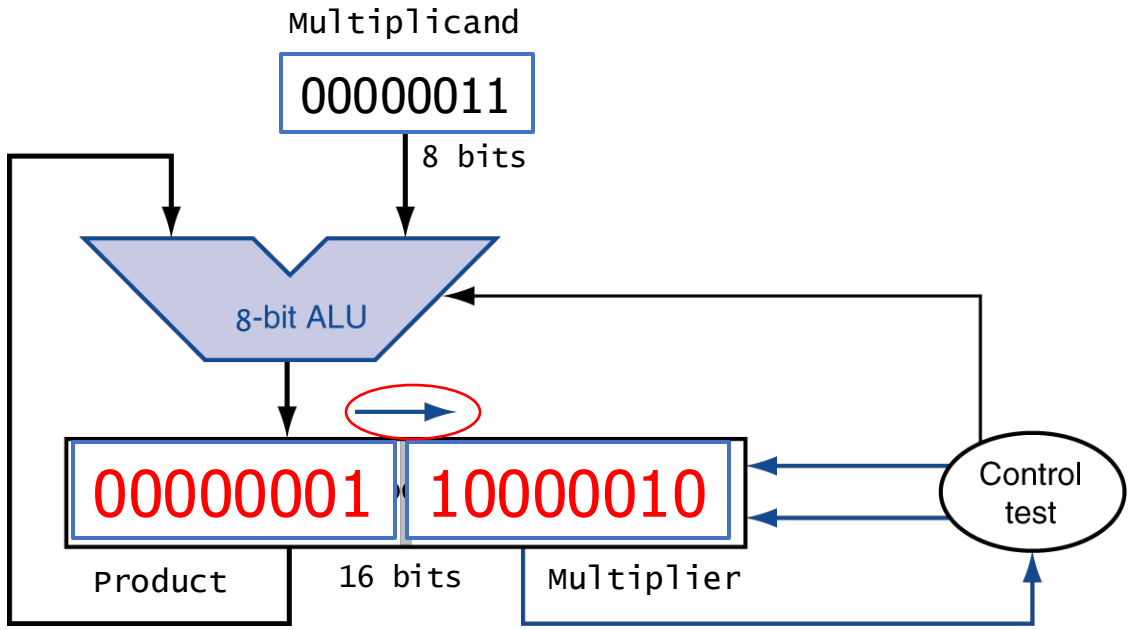
Multiplication Example

- If $\text{Multiplier}_0 = 1$
  - add multiplicand to the left half of product & place the result in the left half of Product register



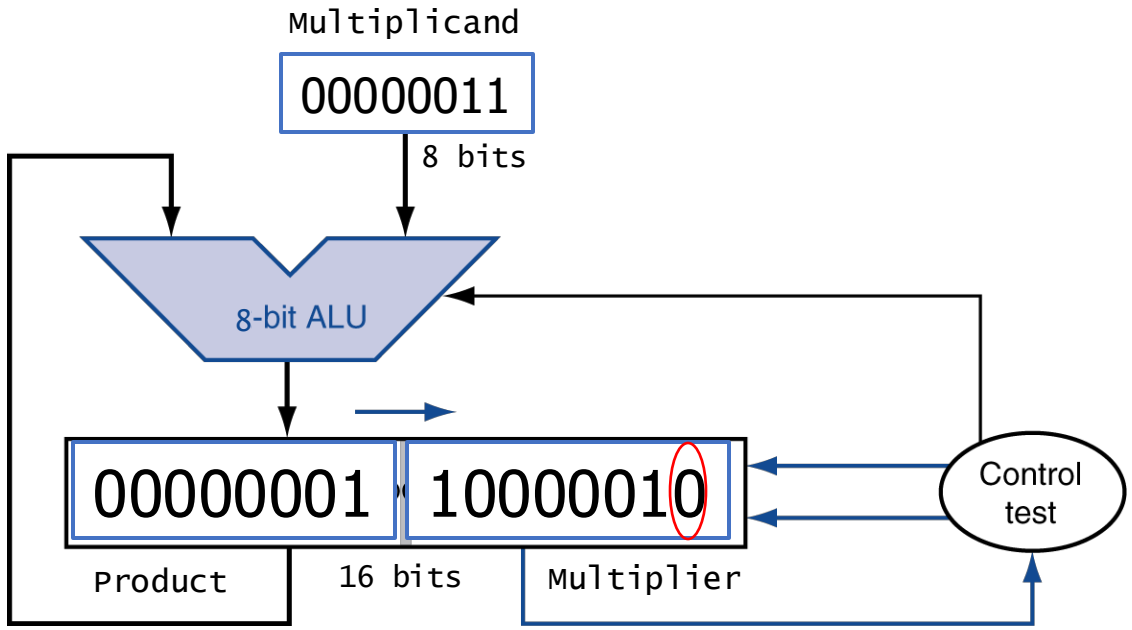
Multiplication Example

- Shift right by 1 bit



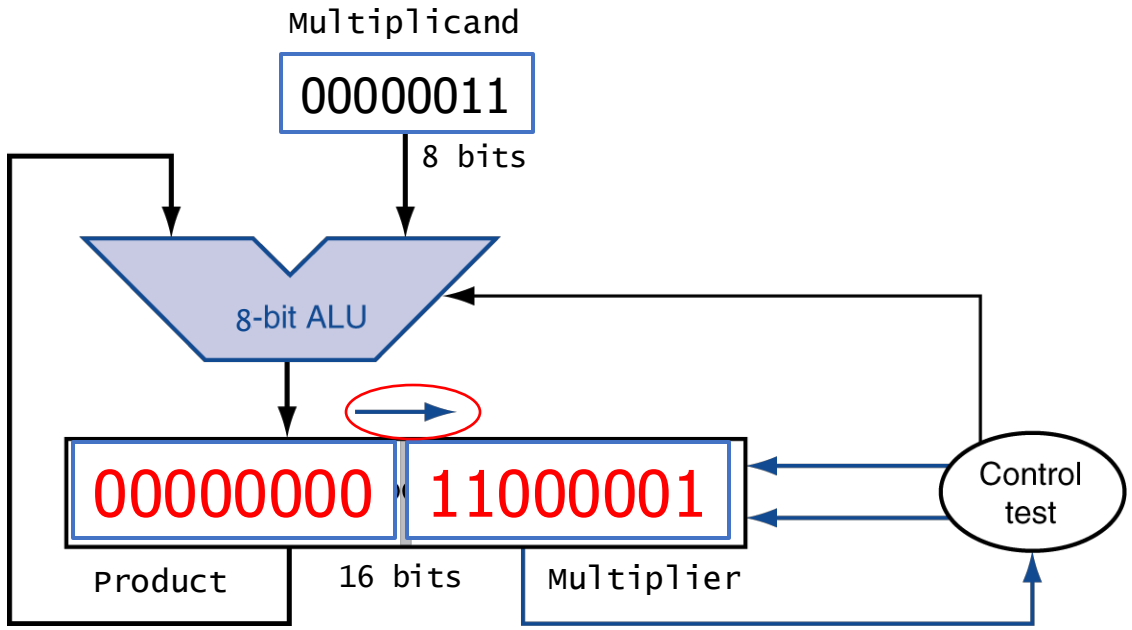
Multiplication Example

- If $\text{Multiplier}_0 = 0$
  - Do not thing



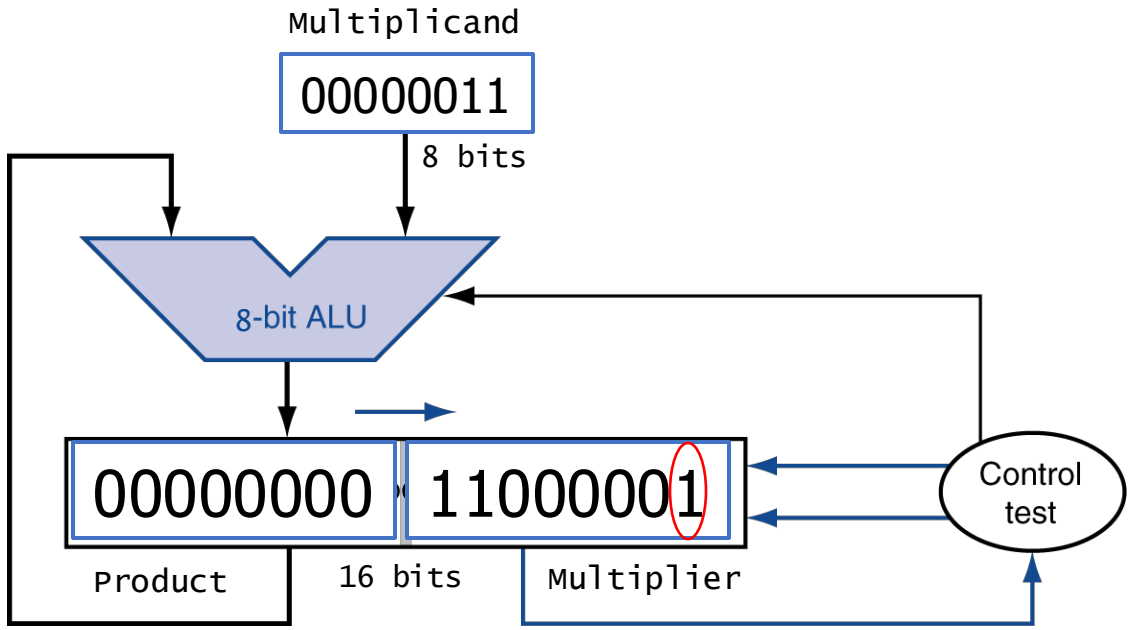
Multiplication Example

- Shift right by 1 bit



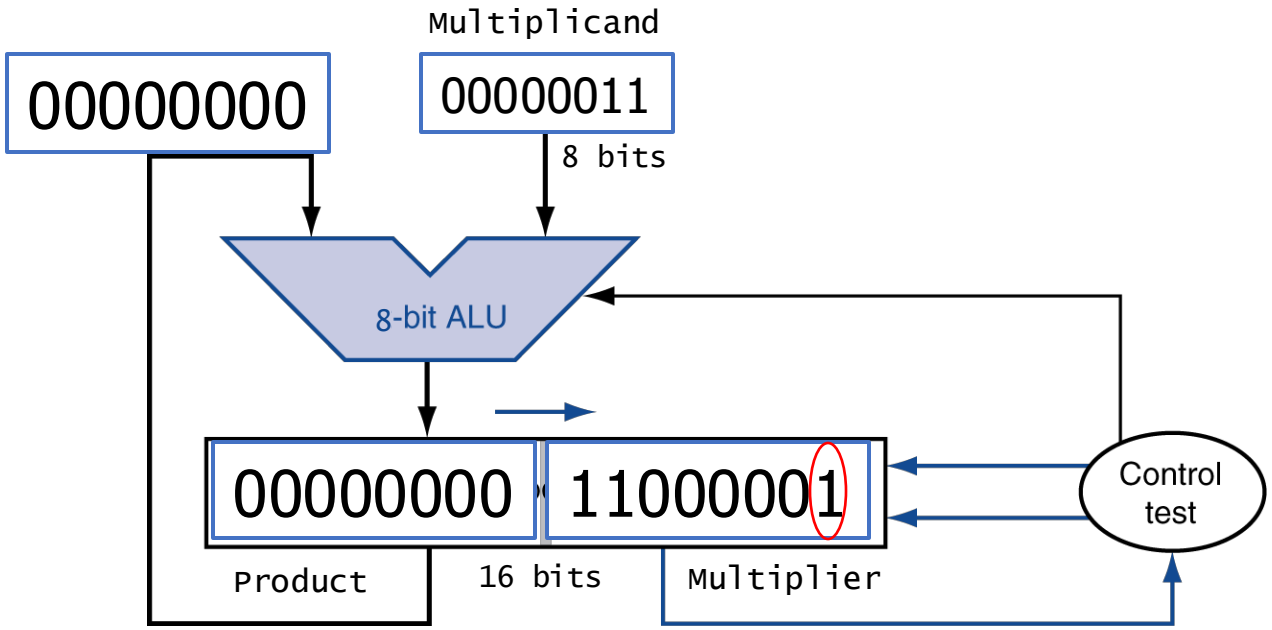
Multiplication Example

- $\text{Multiplier}_0 = 1$



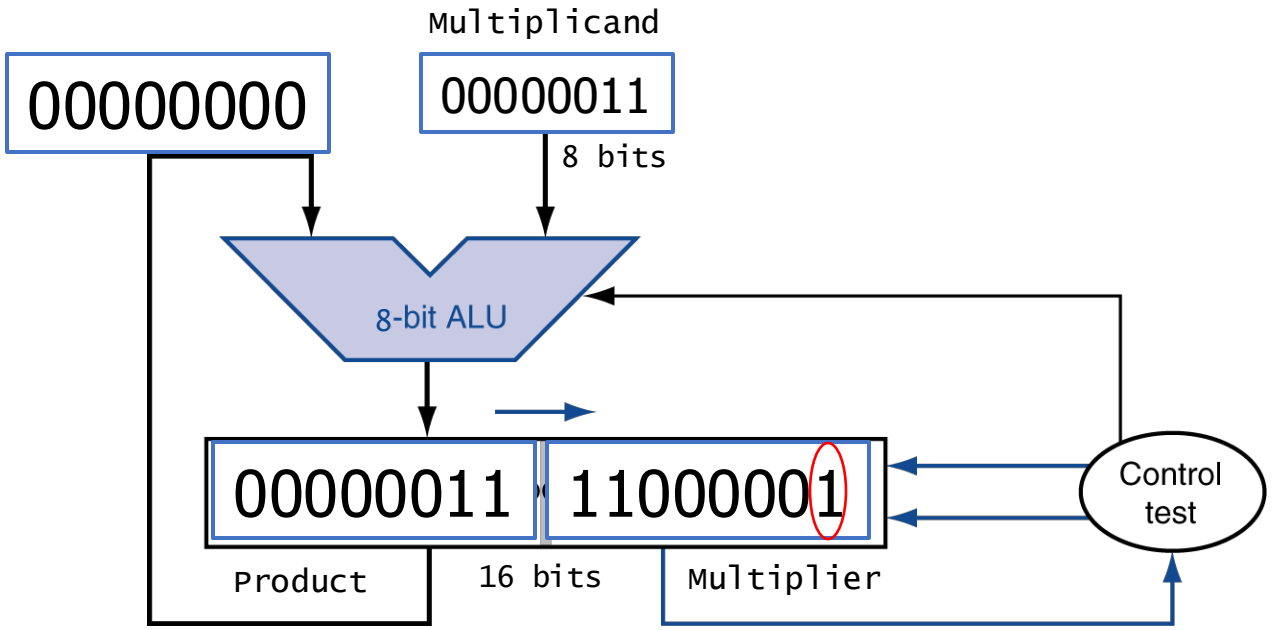
Multiplication Example

- Multiplier$_0$ = 1, add multiplicand



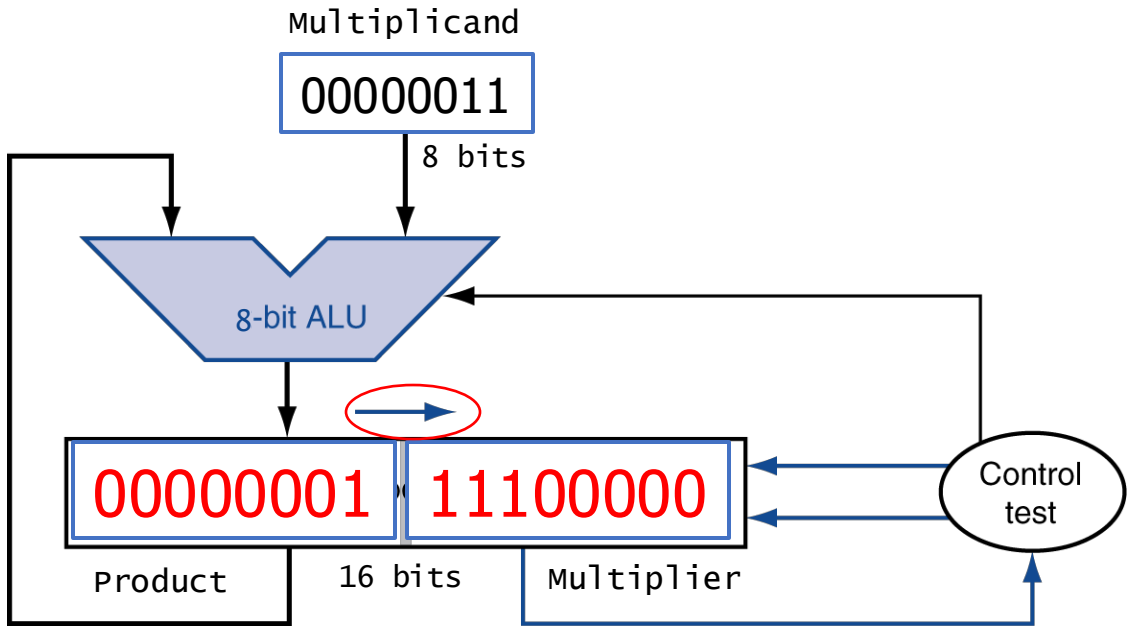
Multiplication Example

- Multiplier$_0$ = 1, add multiplicand



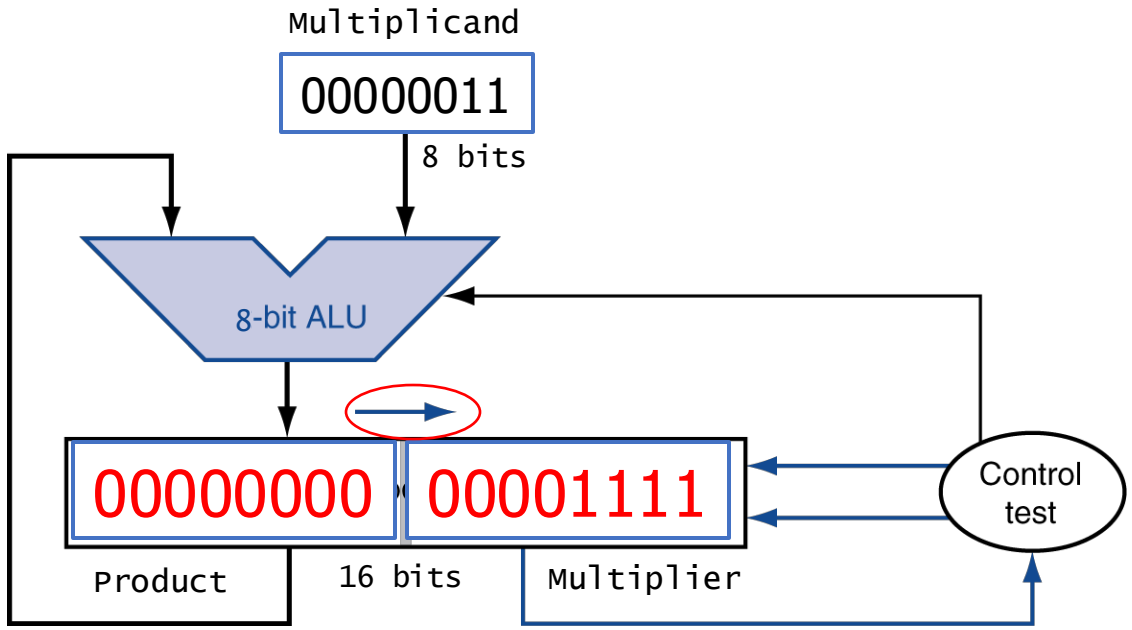
Multiplication Example

- Shift right by 1 bit



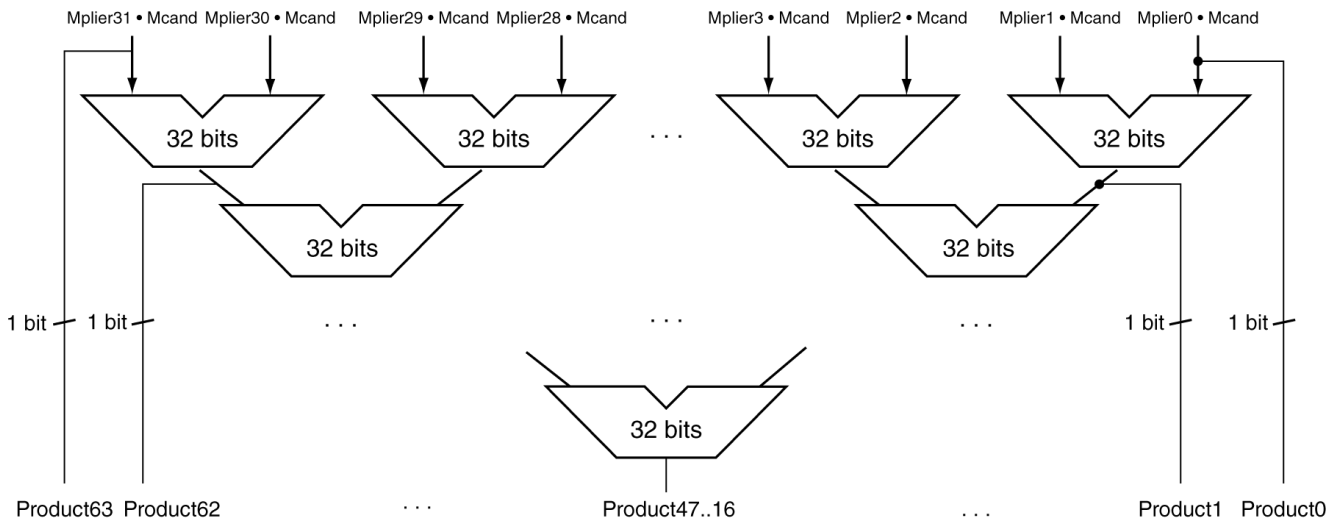
Multiplication Example

- Shift right by 1 bit x 5 times



Faster Multiplier

- Uses multiple adders
  - Cost/performance tradeoff



- Can be pipelined
  - Several multiplication performed in parallel

RISC-V Multiplication

- Four multiply instructions:

```
mul  rd, rs1, rs2
```

  - Multiplies `rs1` and `rs2` and store the lower 32bits of the result in `rd`

```
mulh rd, rs1, rs2
```

  - Multiplies 2 signed operands `rs1` and `rs2` and stores the upper 32 bits in `rd`

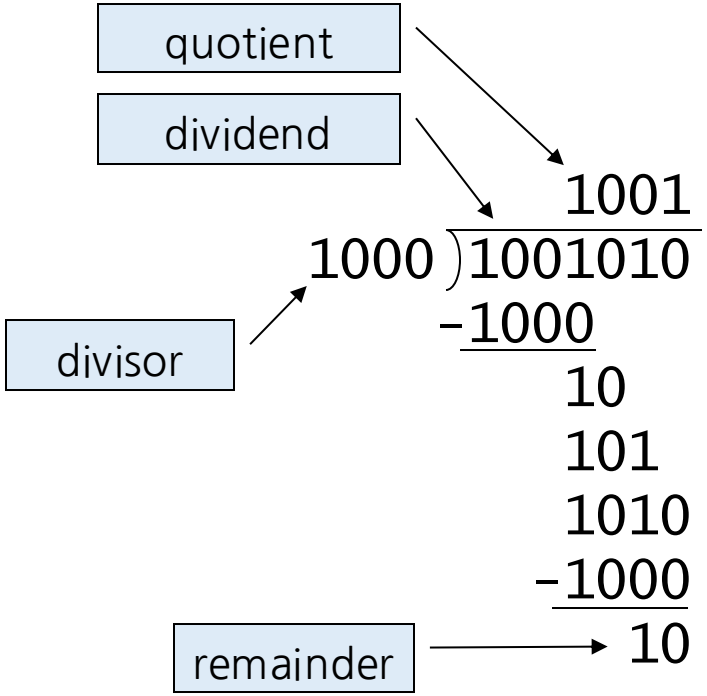
```
mulhu rd, rs1, rs2
```

  - Multiplies 2 unsigned operands `rs1` and `rs2` and stores the upper 32 bits in `rd`

```
mulhsu rd, rs1, rs2
```

  - Multiplies signed operand `rs1` and unsigned operand `rs2` and stores the upper 32 bits in `rd`

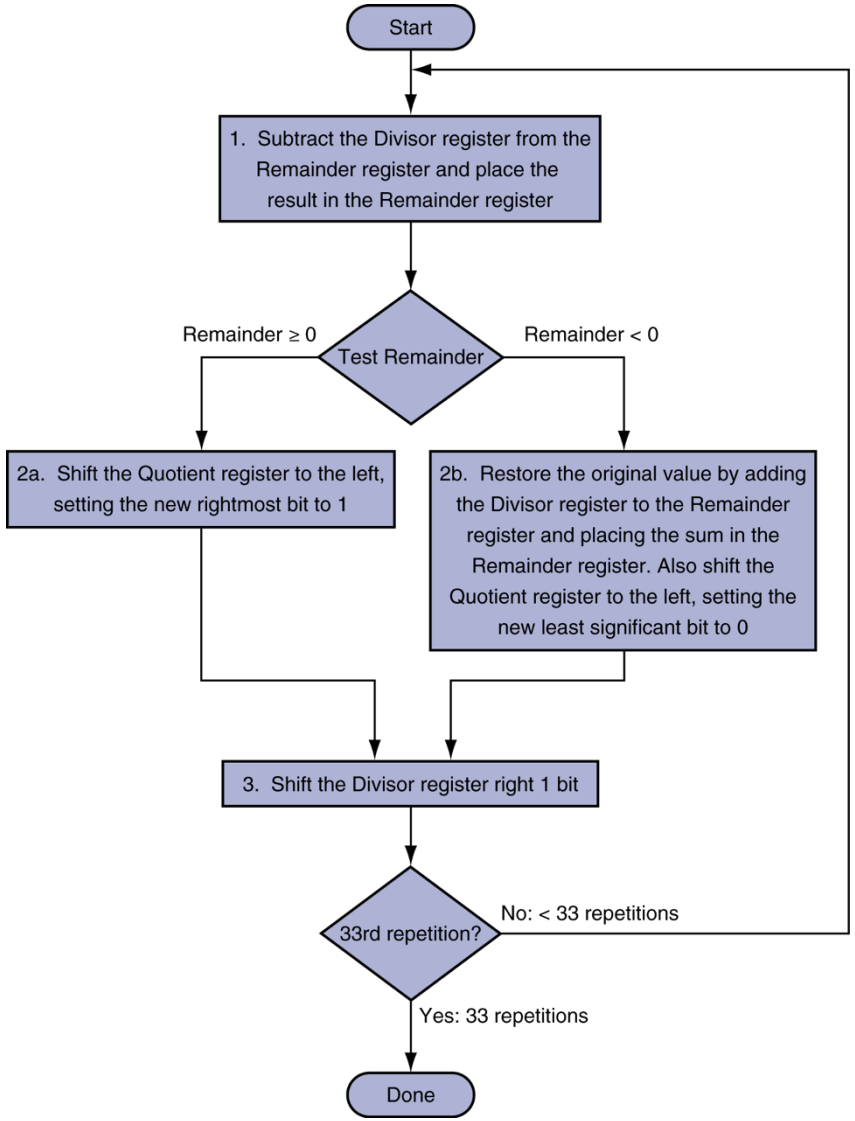
Division

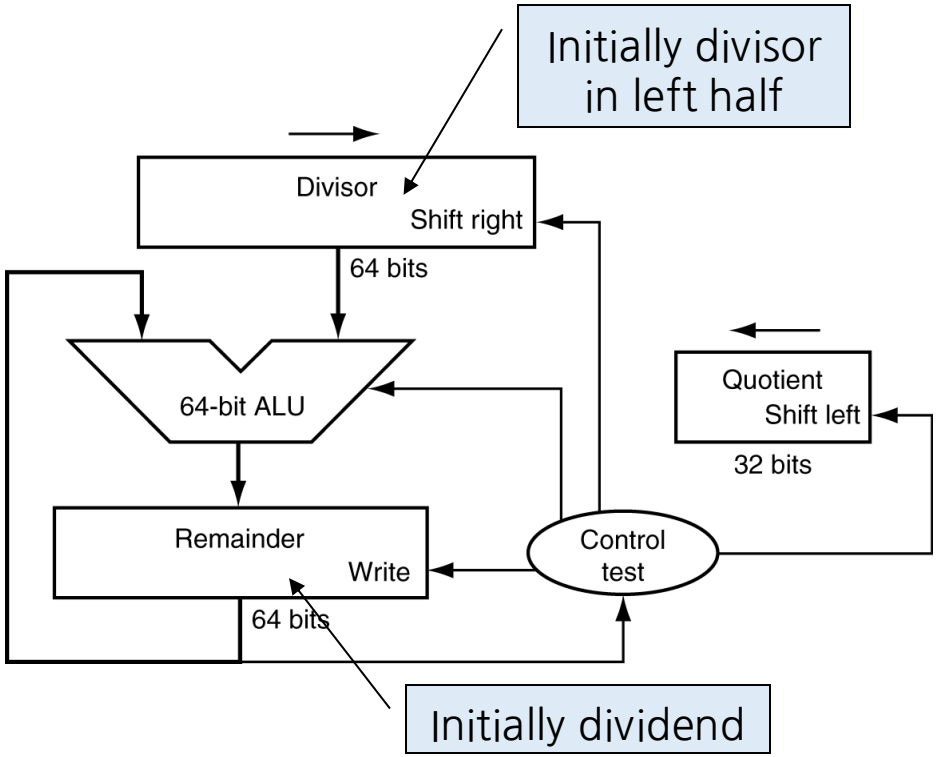


n-bit operands yield *n*-bit quotient and remainder

- Check for 0 divisor —> if 0 raise an exception
- If divisor ≤ dividend bits
  - 1 bit in quotient, subtract
- Otherwise
  - 0 bit in quotient, bring down next dividend bit
- Restoring division
  - Do the subtract, and if remainder goes < 0, add divisor back

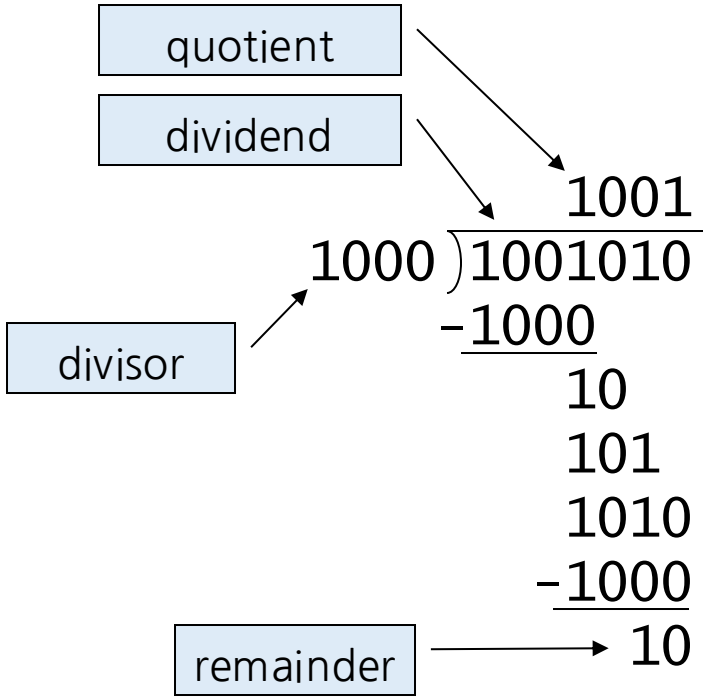
Division Hardware

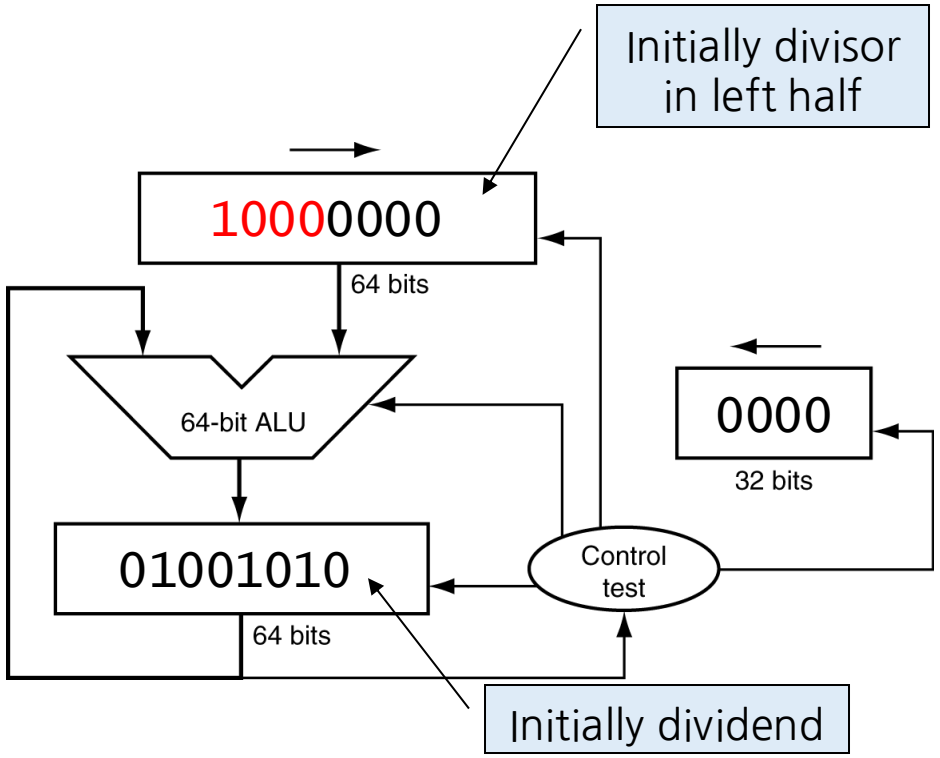




Division Hardware: Example

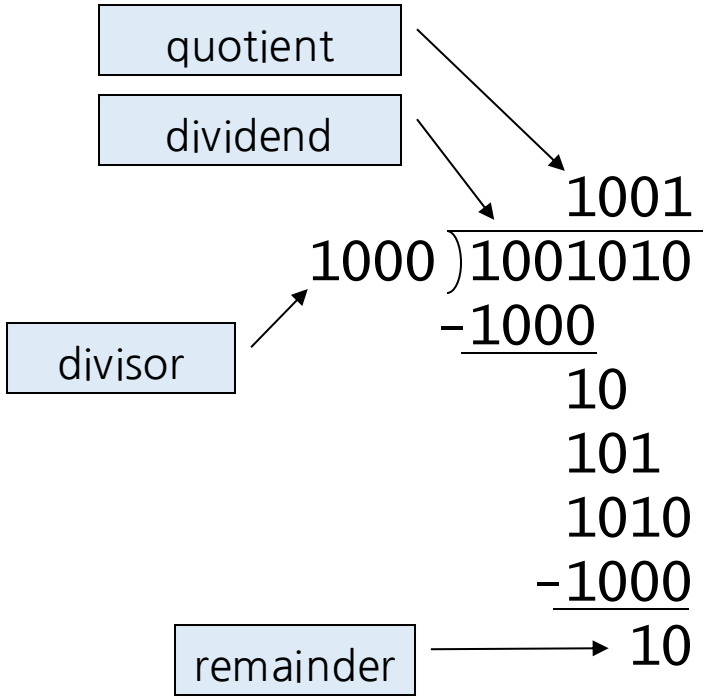
- Initially

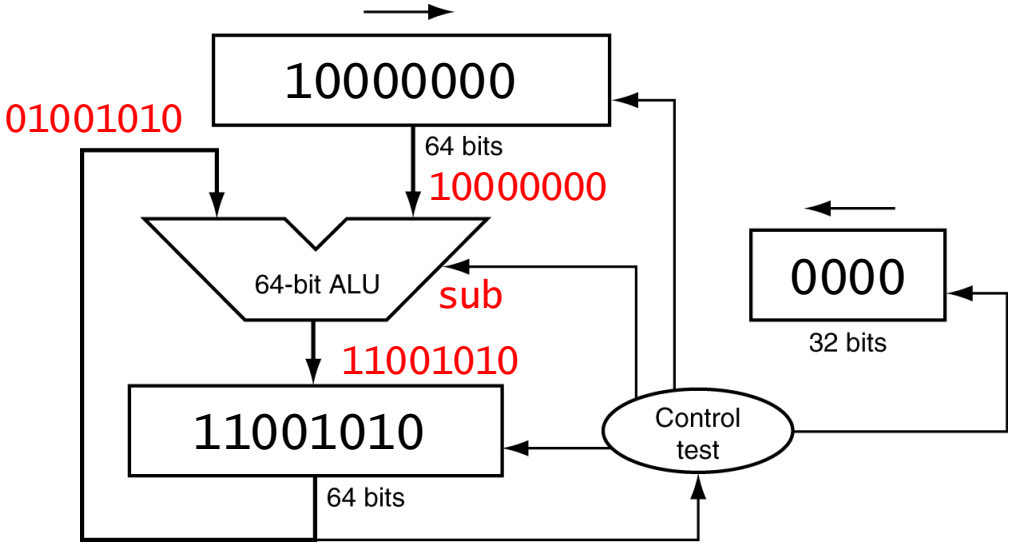




Division Hardware: Example

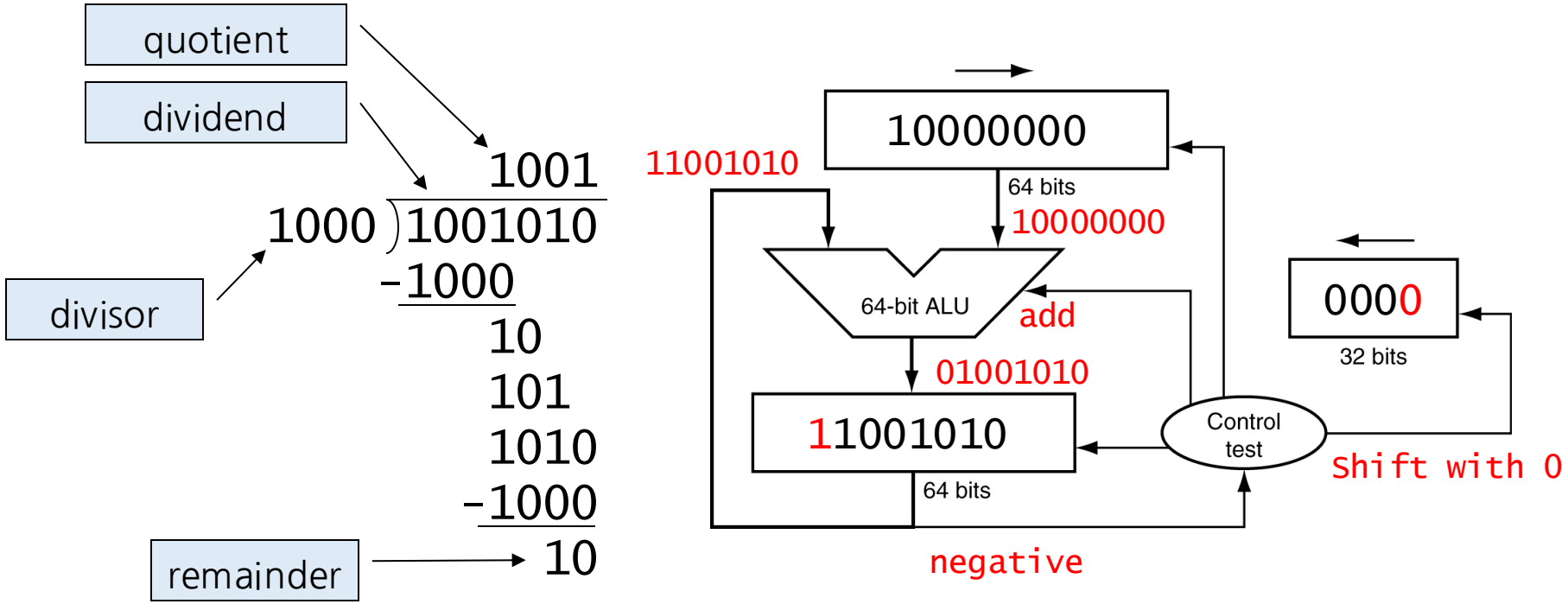
- Iteration 1-1





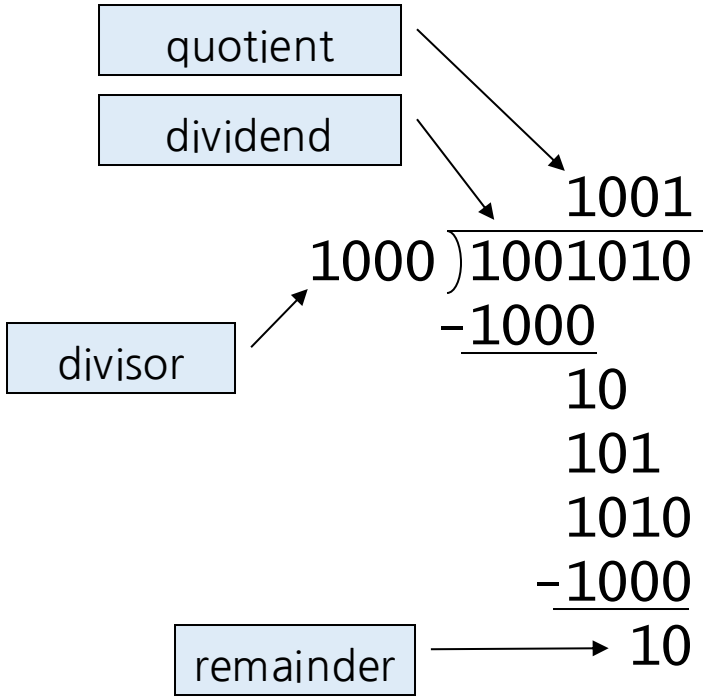
Division Hardware: Example

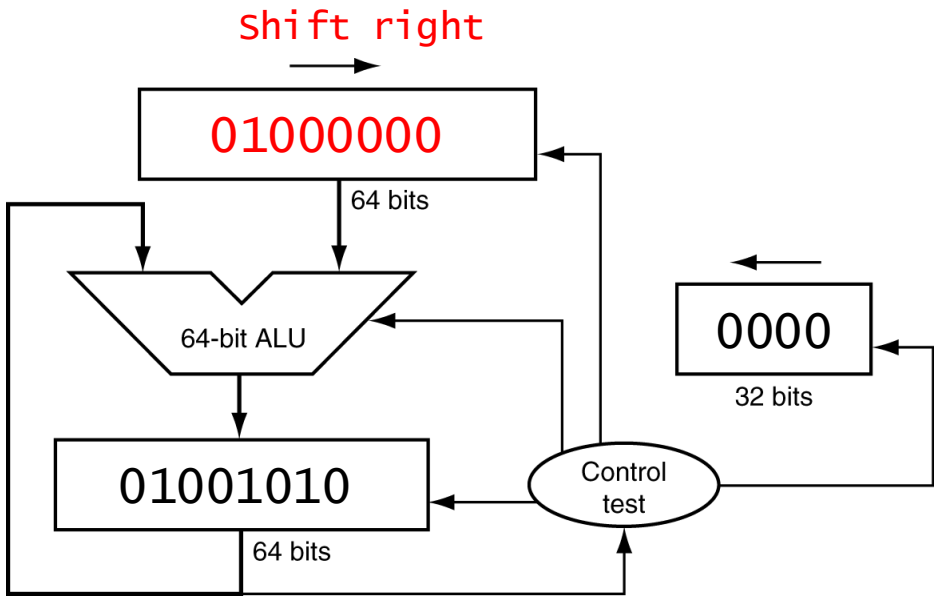
- Iteration 1-2b



Division Hardware: Example

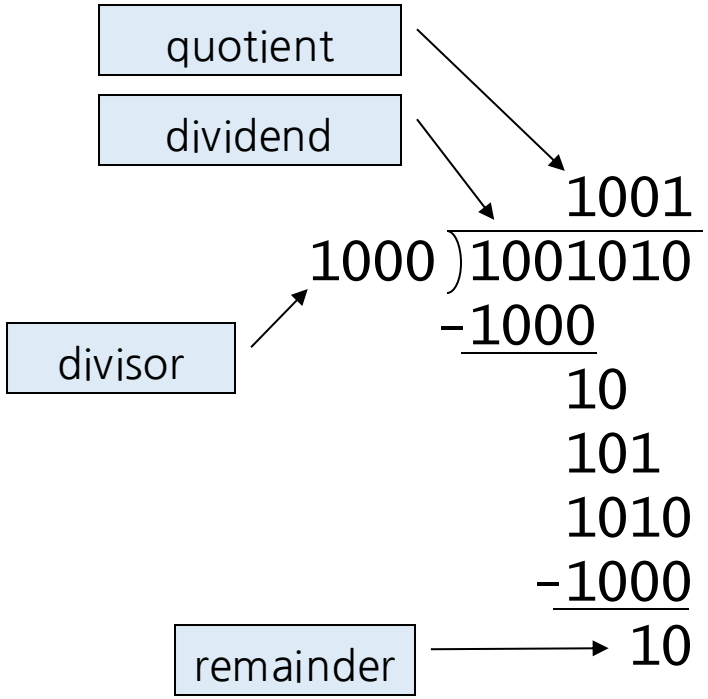
- Iteration 1-3

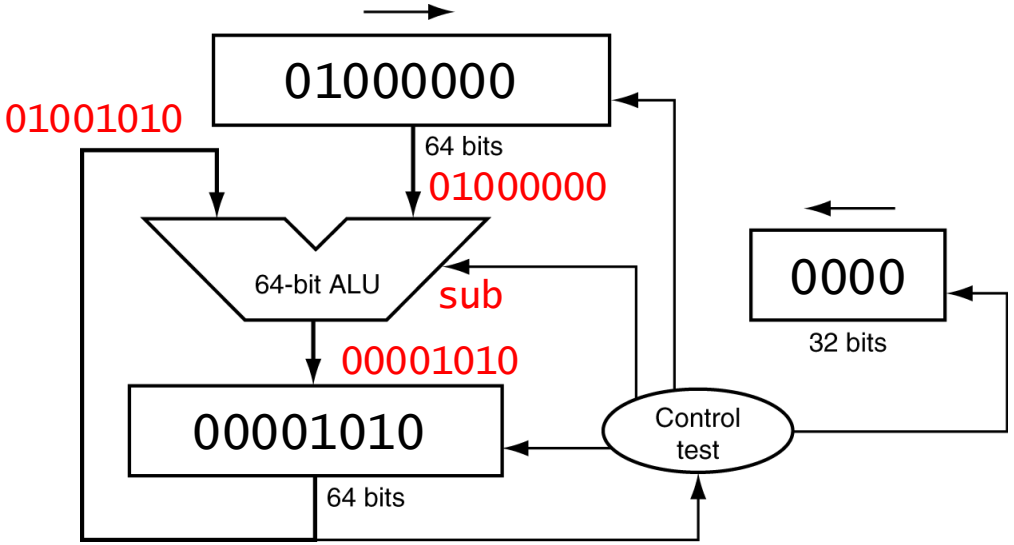




Division Hardware: Example

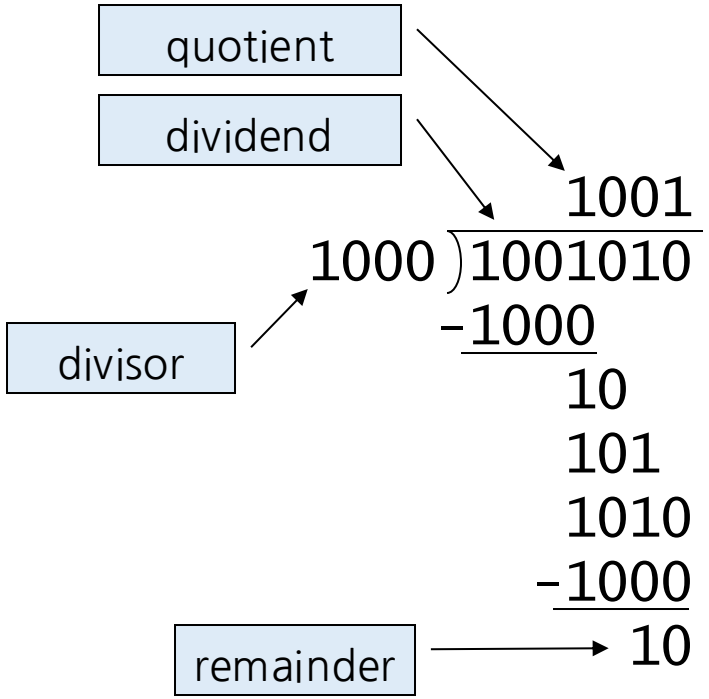
- Iteration 2-1

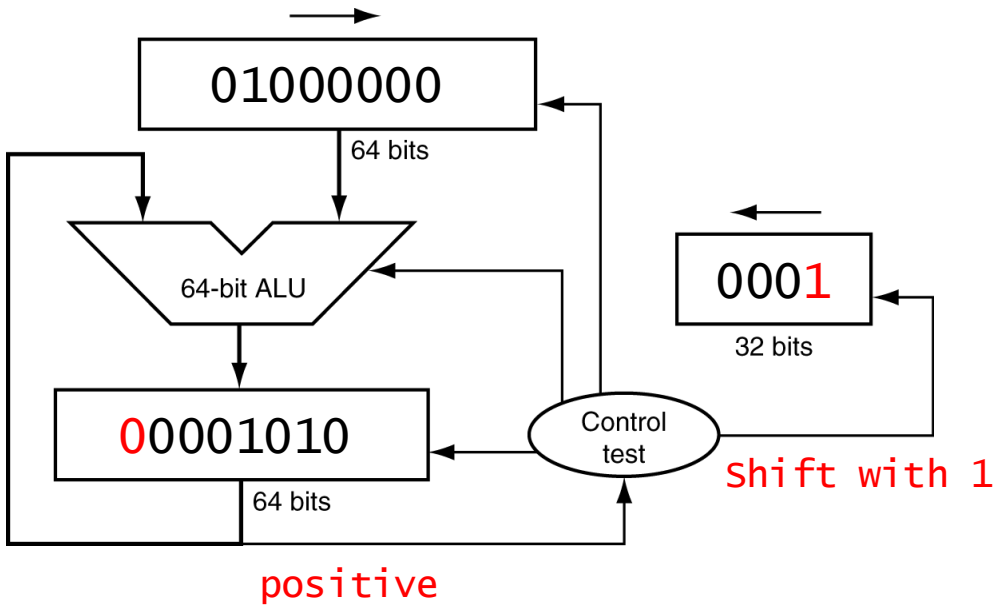




Division Hardware: Example

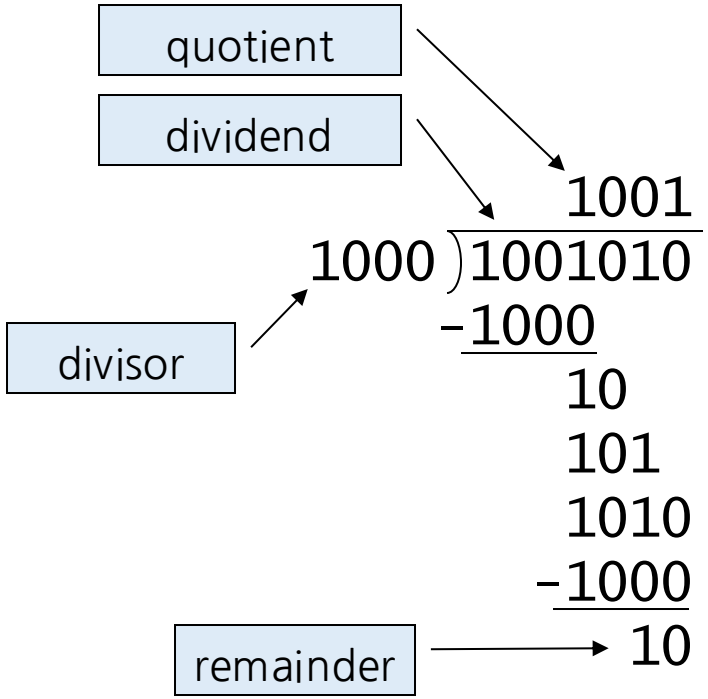
- Iteration 2-2a

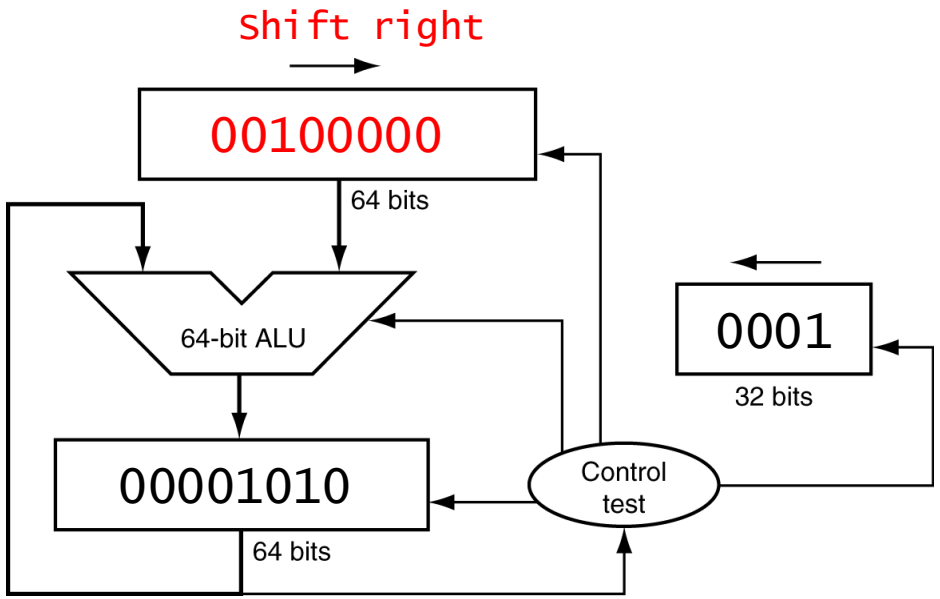




Division Hardware: Example

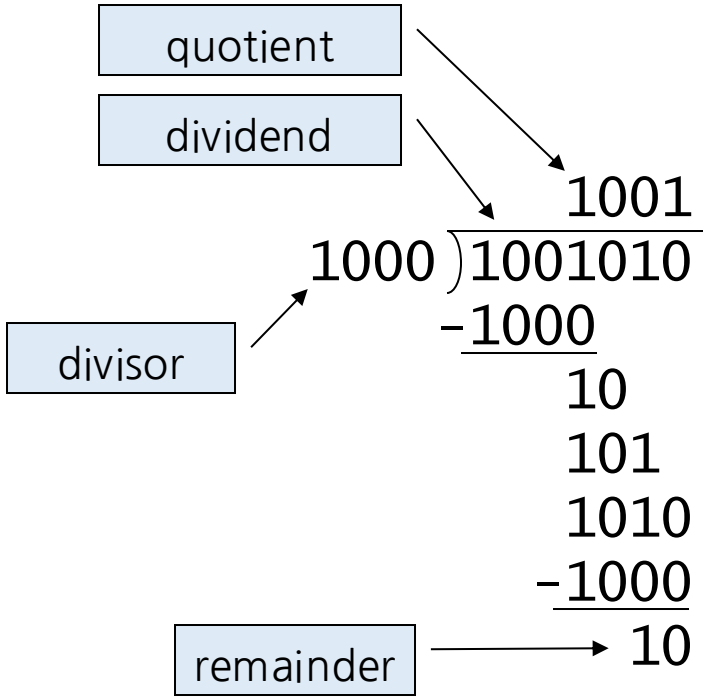
- Iteration 2-3

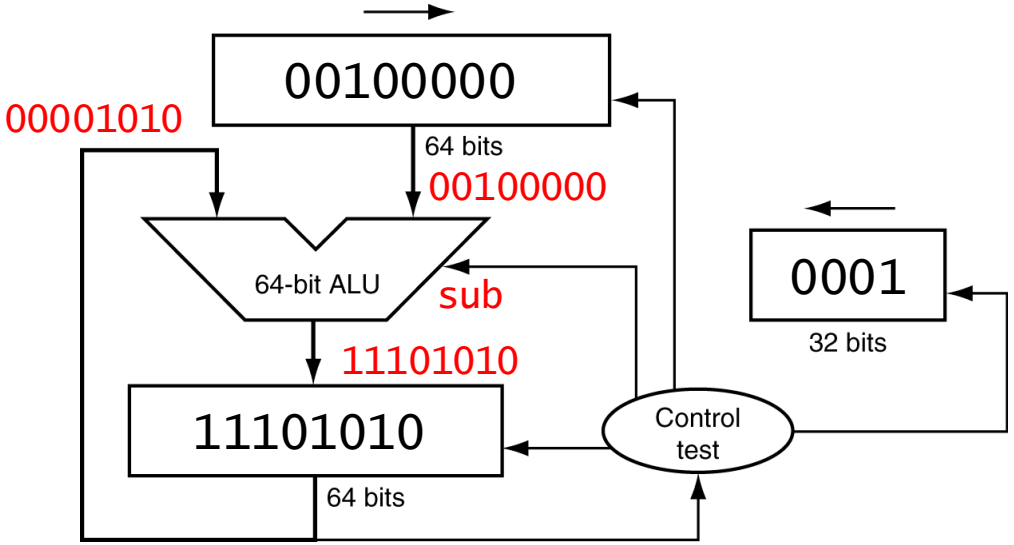




Division Hardware: Example

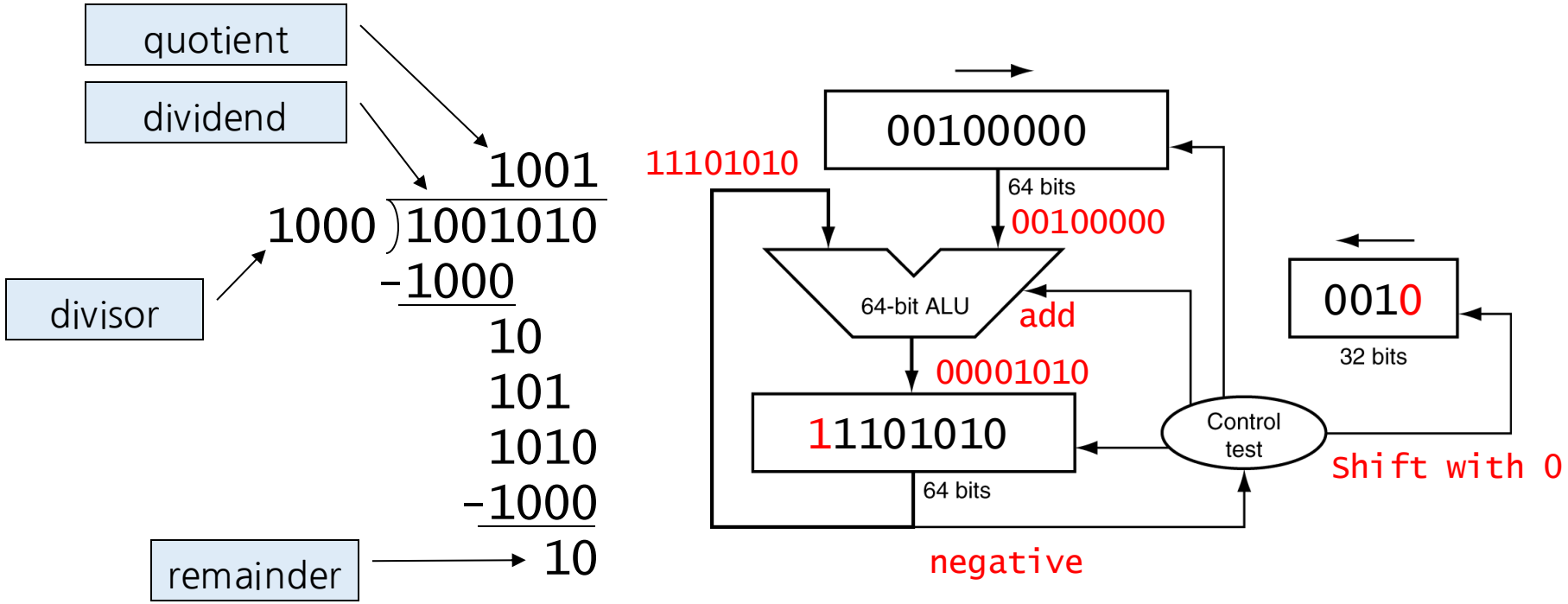
- Iteration 3-1





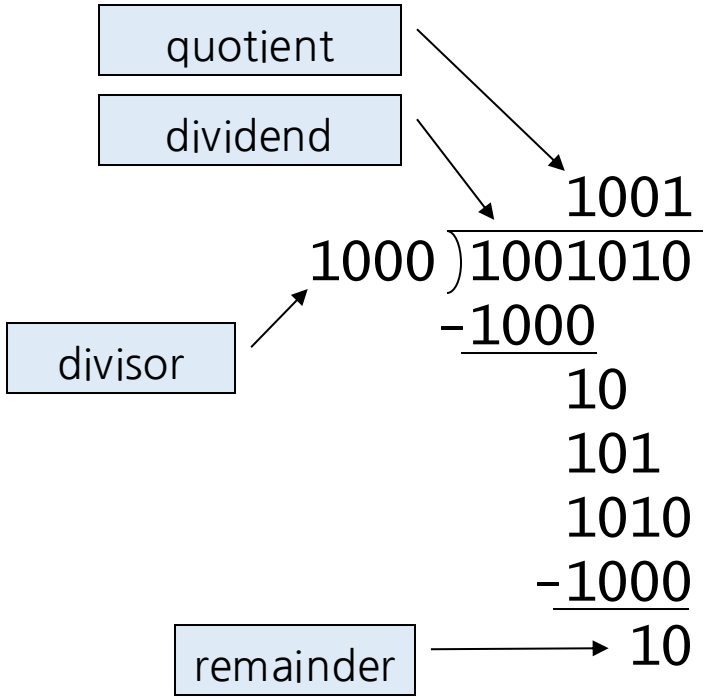
Division Hardware: Example

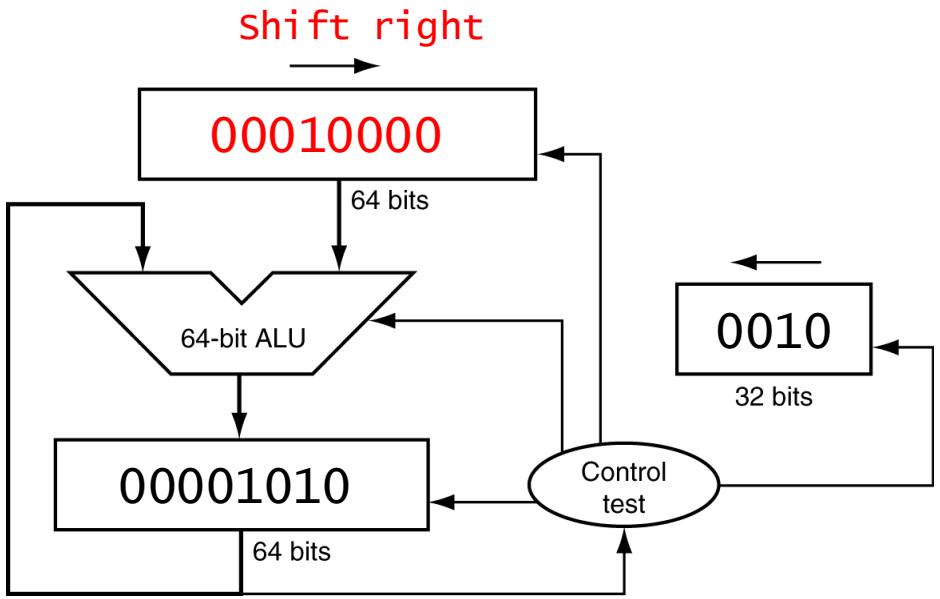
- Iteration 3-2b



Division Hardware: Example

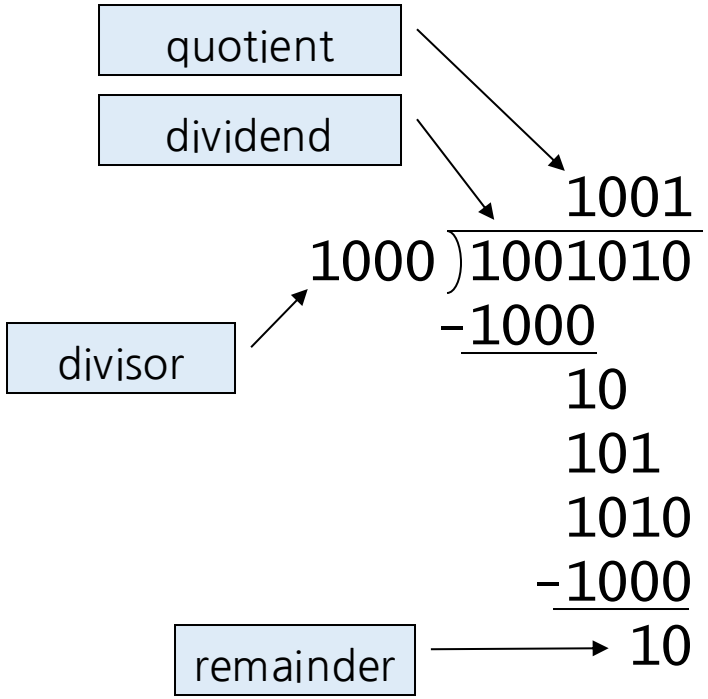
- Iteration 3-3

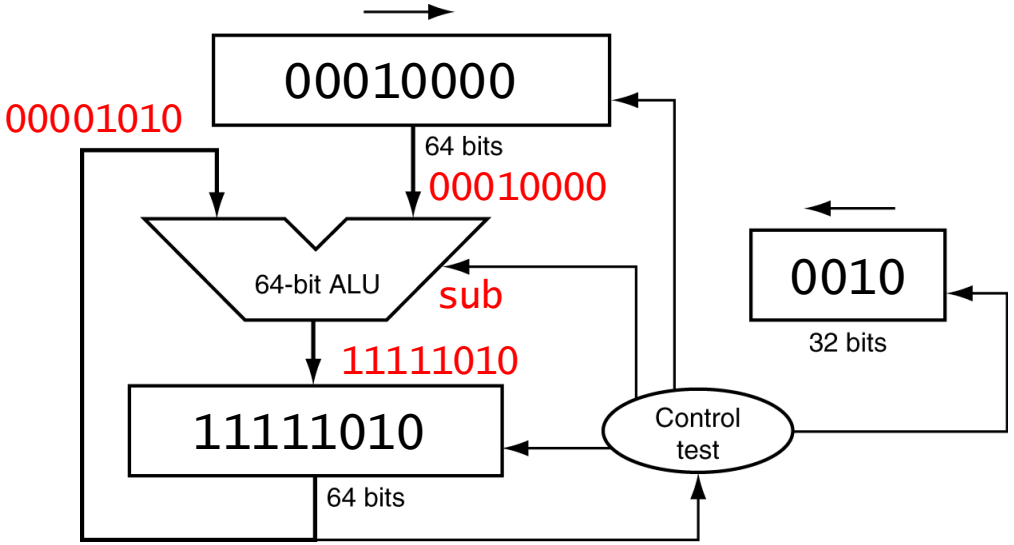




Division Hardware: Example

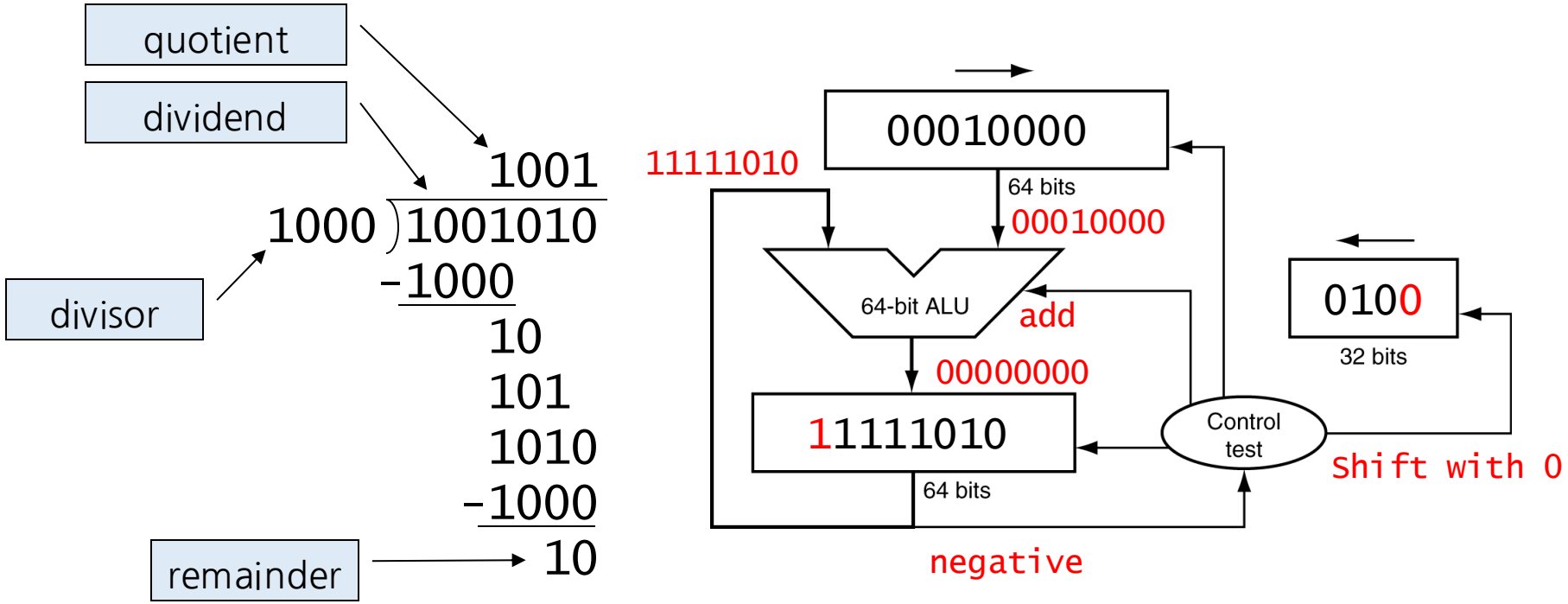
- Iteration 4-1





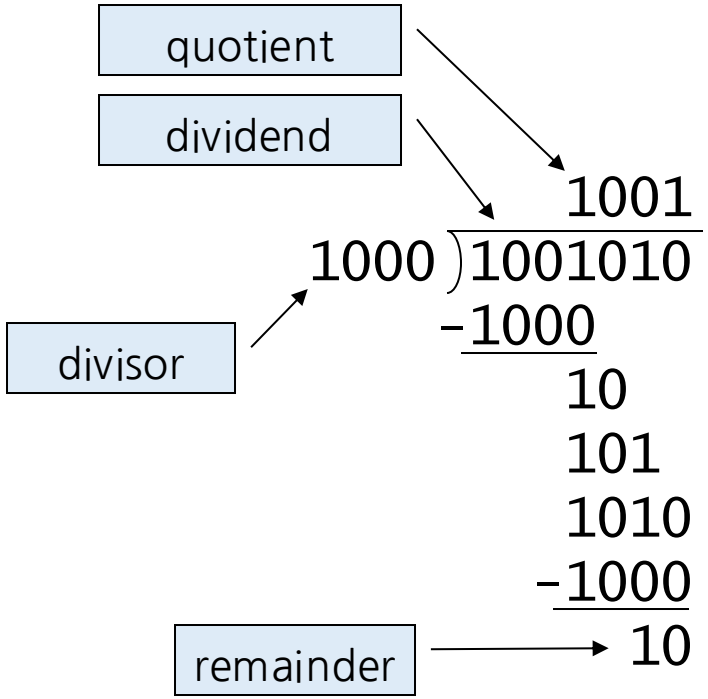
Division Hardware: Example

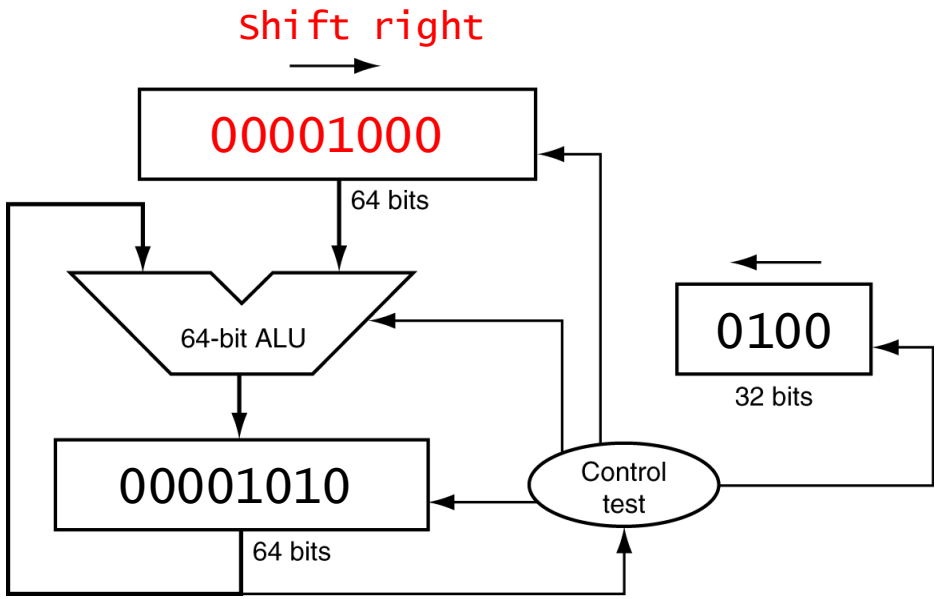
- Iteration 4-2b



Division Hardware: Example

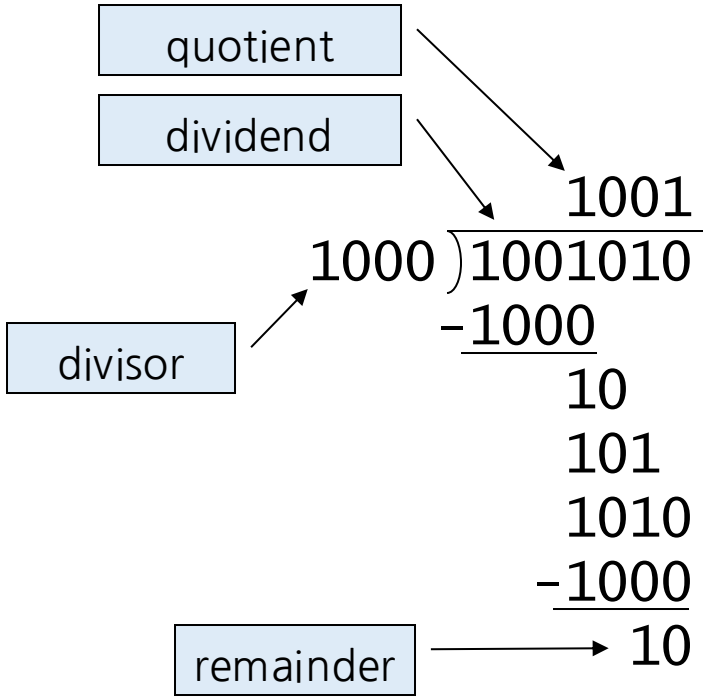
- Iteration 4-3

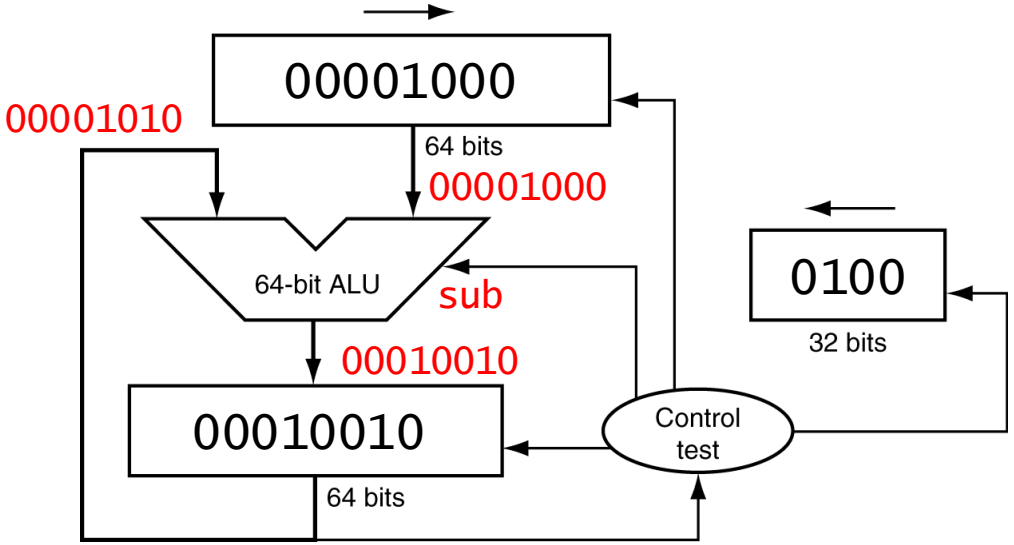




Division Hardware: Example

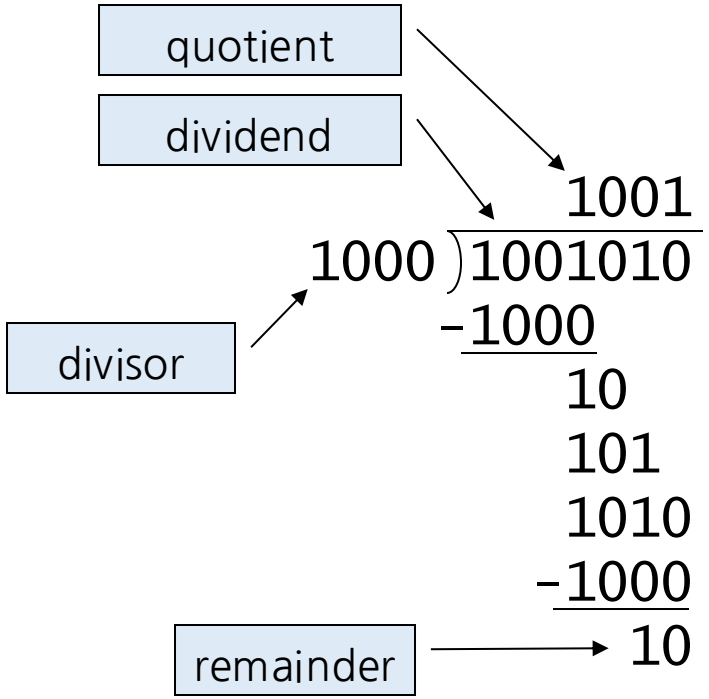
- Iteration 5-1

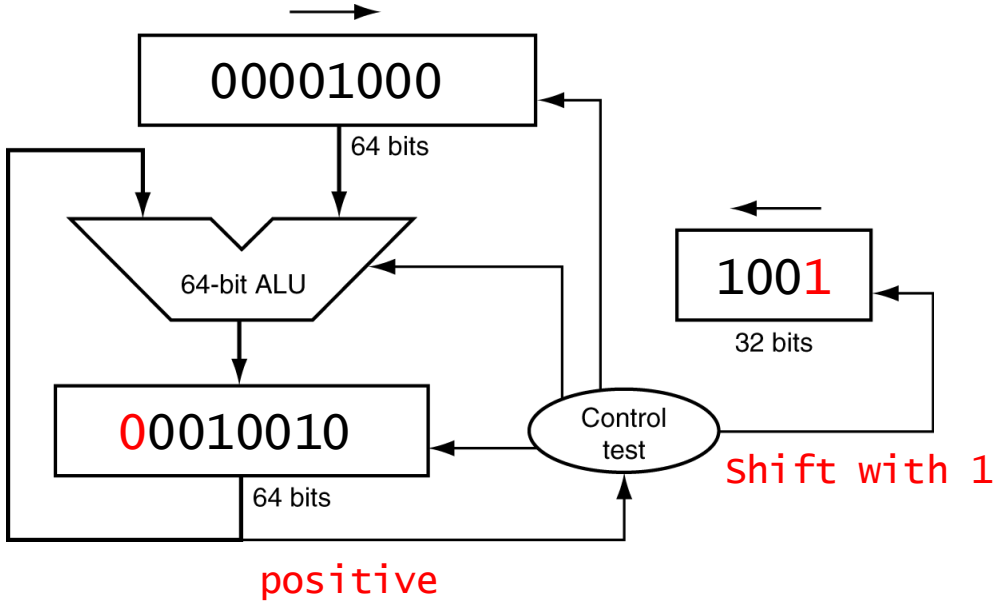




Division Hardware: Example

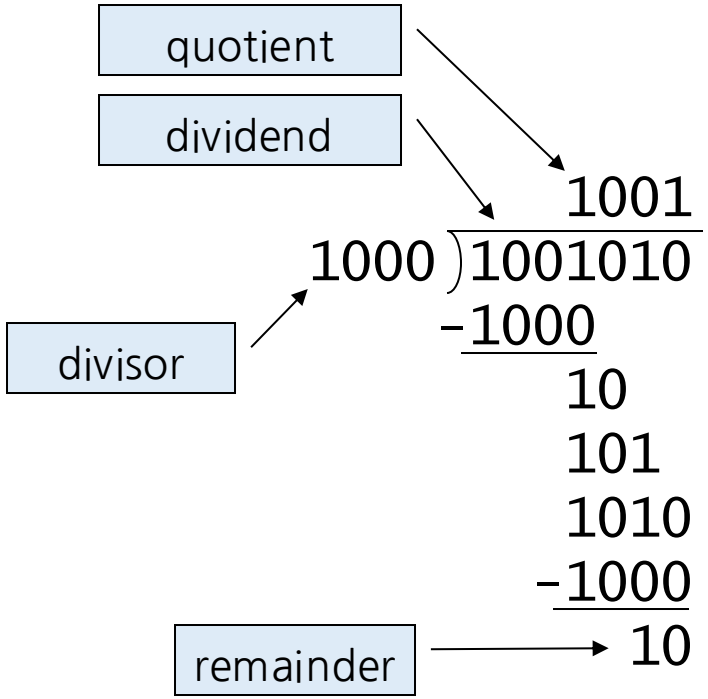
- Iteration 5-2a

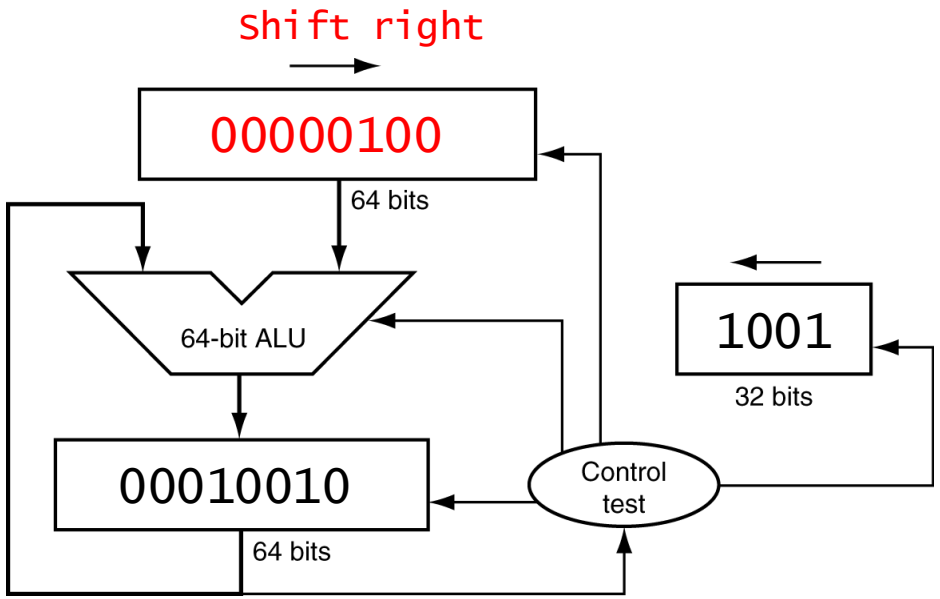




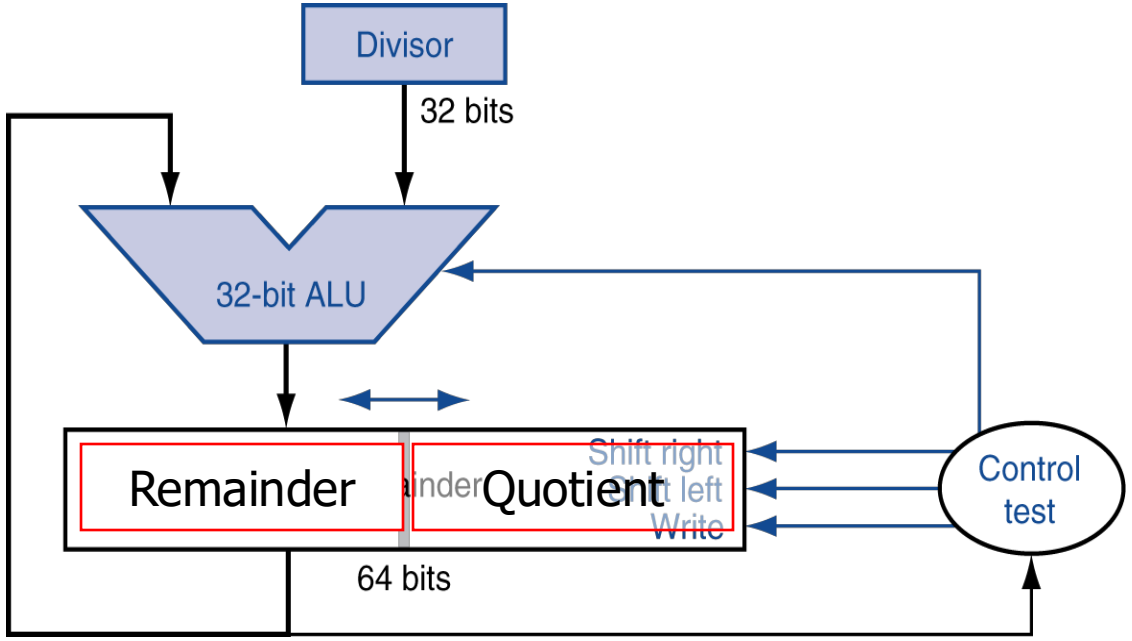
Division Hardware: Example

- Iteration 5-3





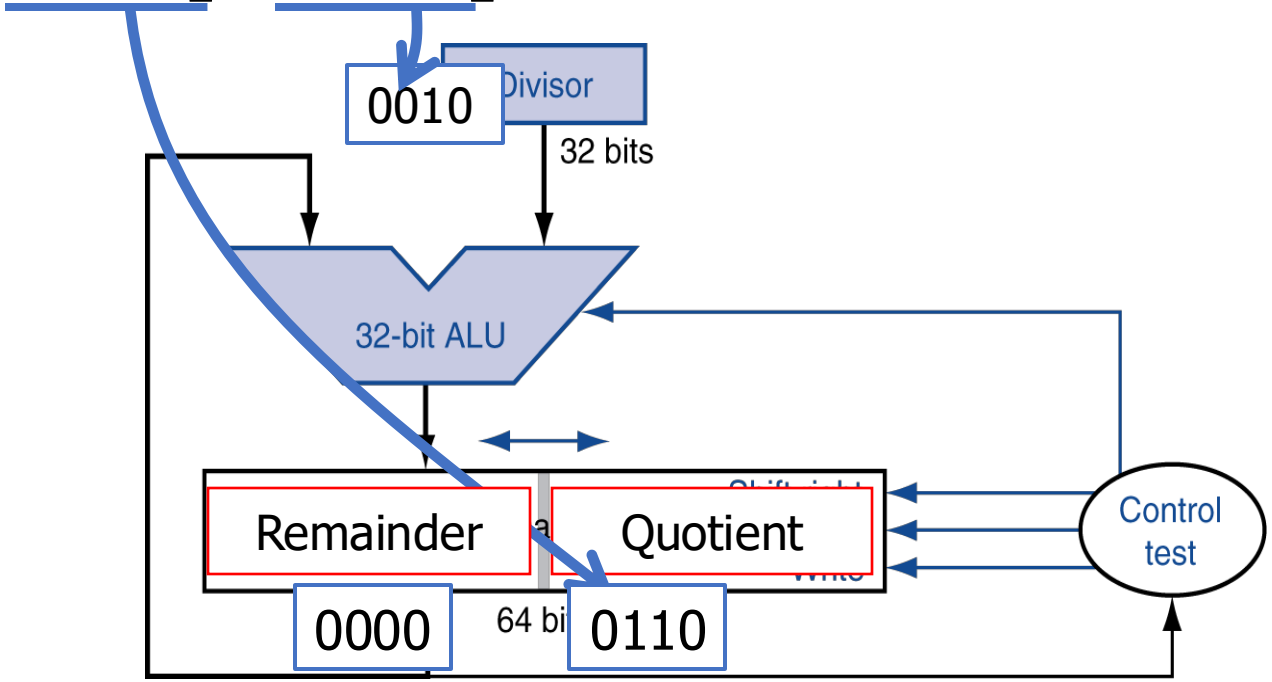
Optimized Divider



- Looks a lot like a multiplier!
  - Same hardware can be used for both
- One cycle per partial-remainder subtraction

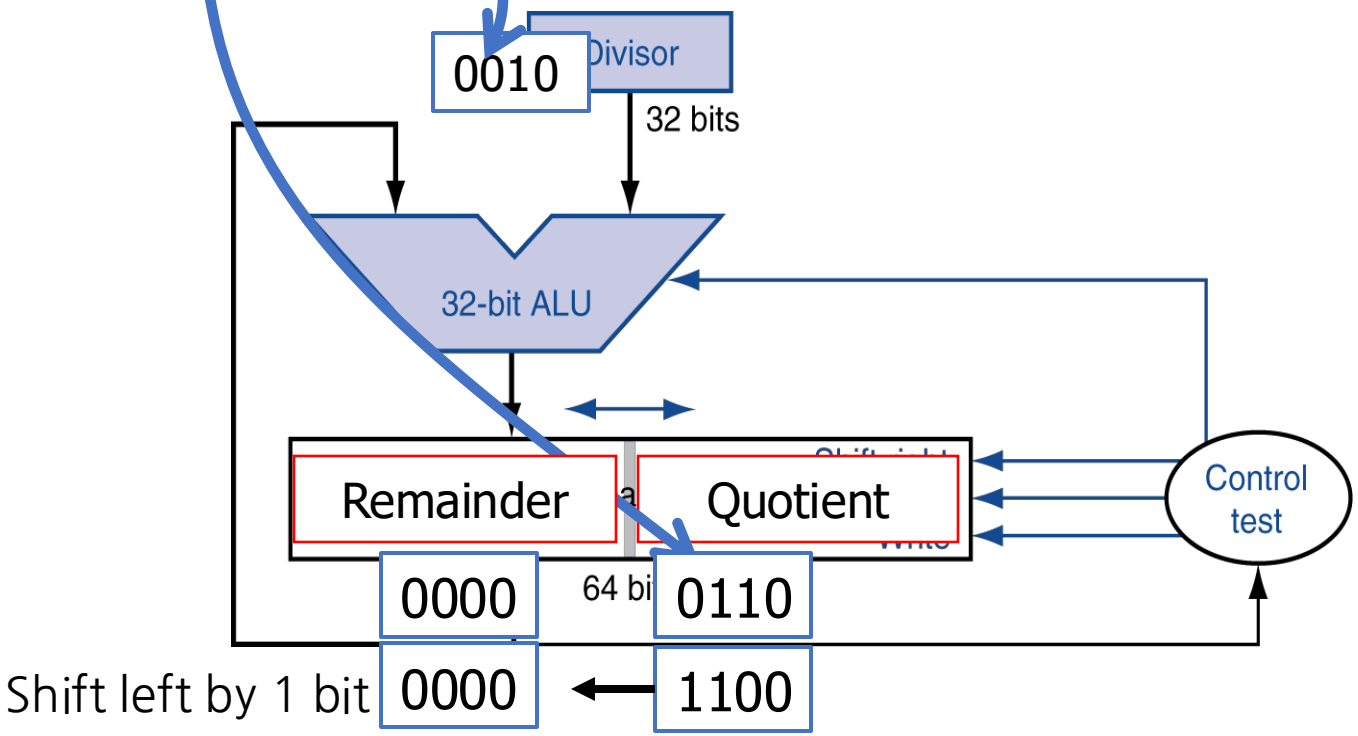
Division Example

- $6_{10} / 2_{10} = 0110_2 / 0010_2$



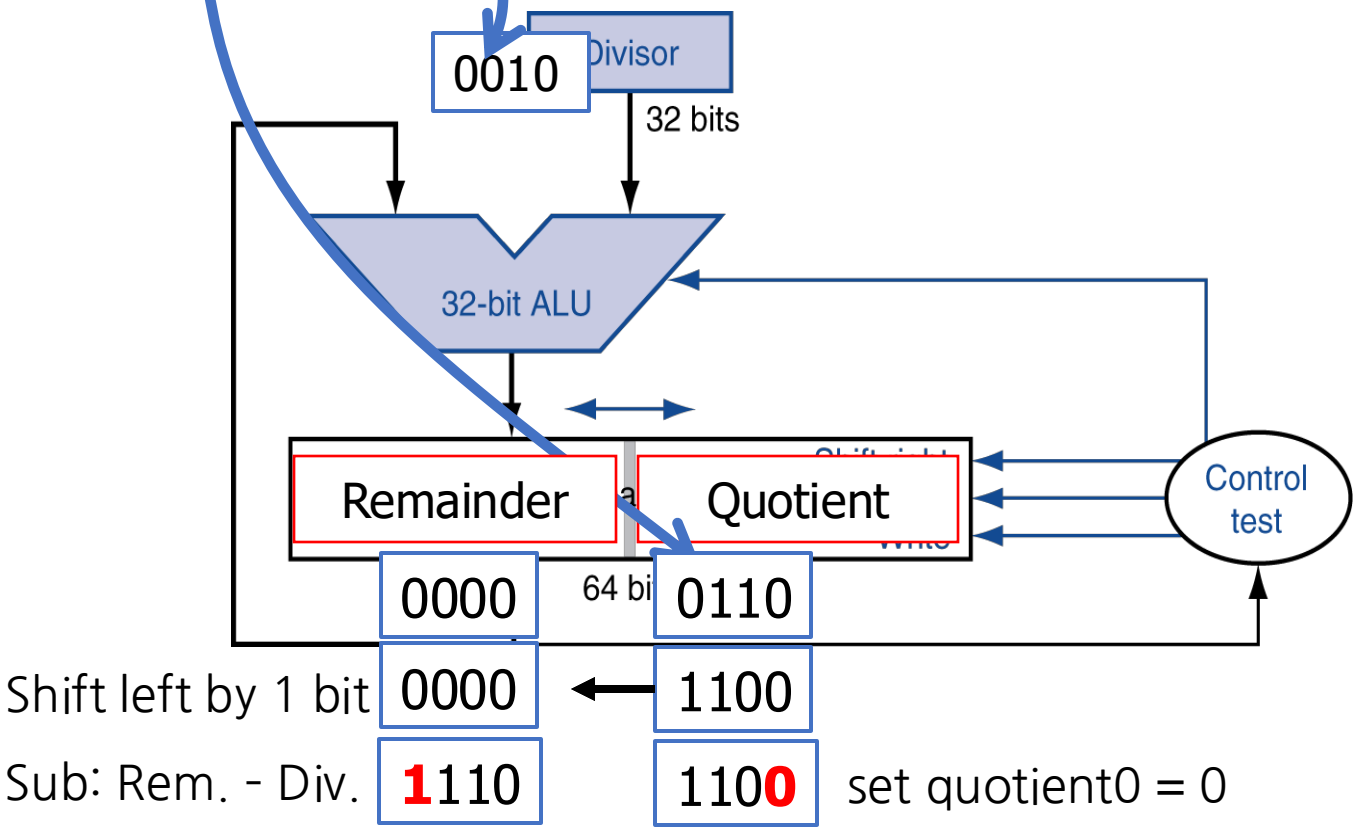
Division Example

- $6_{10} / 2_{10} = 0110_2 / 0010_2$



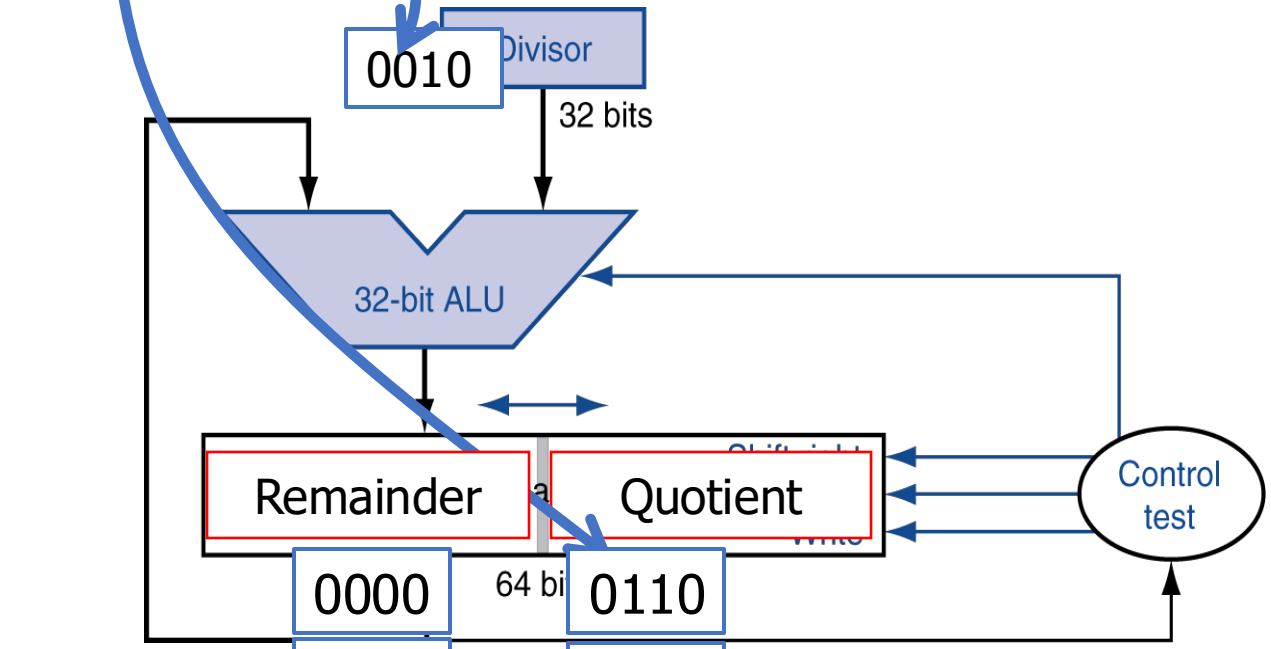
Division Example

- $6_{10} / 2_{10} = 0110_2 / 0010_2$



Division Example

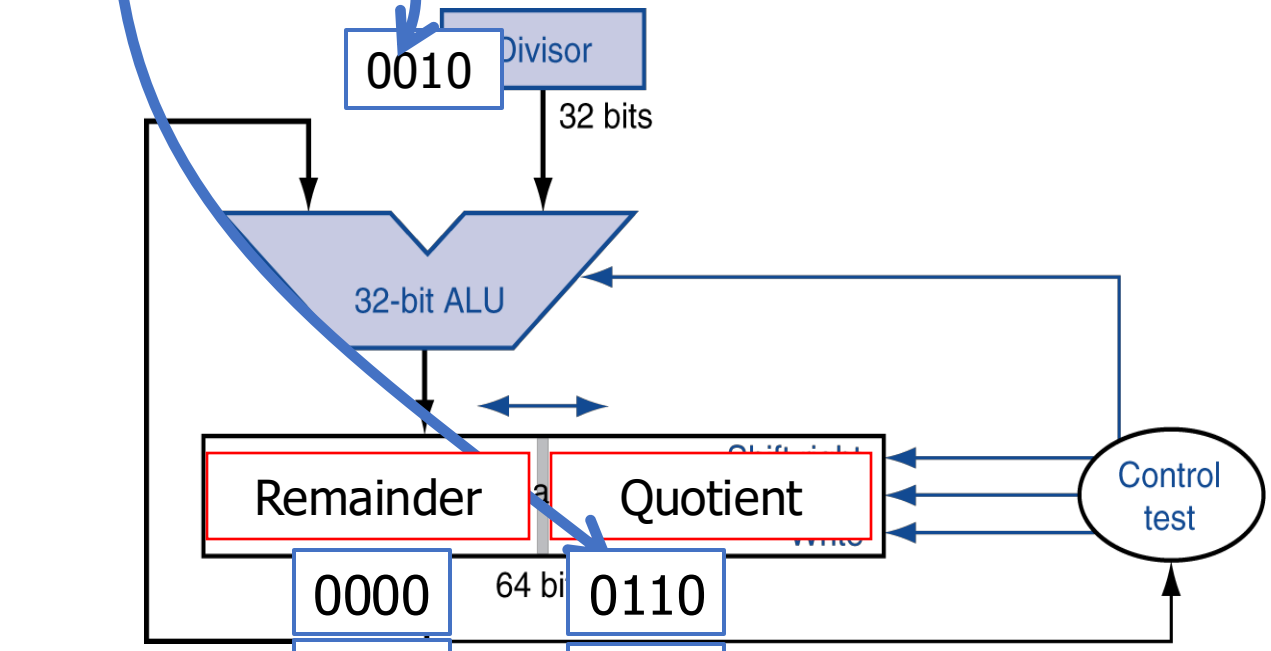
- $6_{10} / 2_{10} = 0110_2 / 0010_2$



	0000	0110
Shift left by 1 bit	0000	1100
Sub: Rem. - Div.	1110	1100
Restore rem.	0000	1100

Division Example

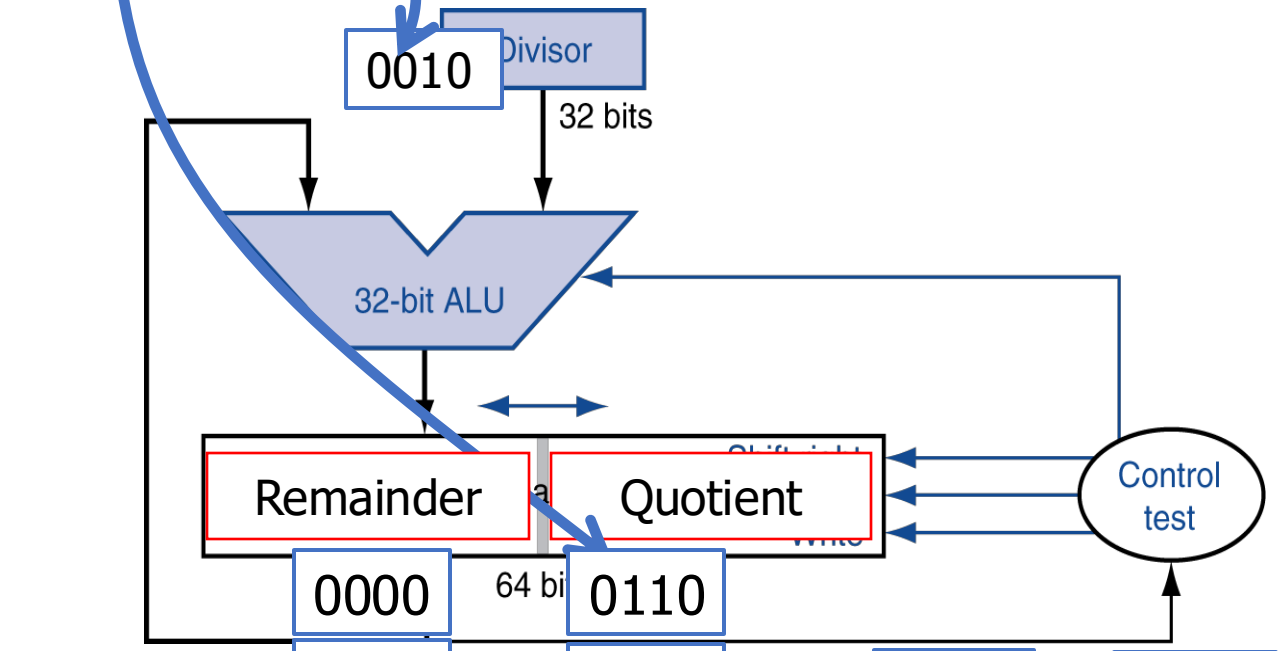
- $6_{10} / 2_{10} = 0110_2 / 0010_2$



	0000		0110
Shift left by 1 bit	0000	←	1100
Sub: Rem. - Div.	1110		1100
Restore rem.	0000		1100
Shift left by 1 bit	0001	←	1000

Division Example

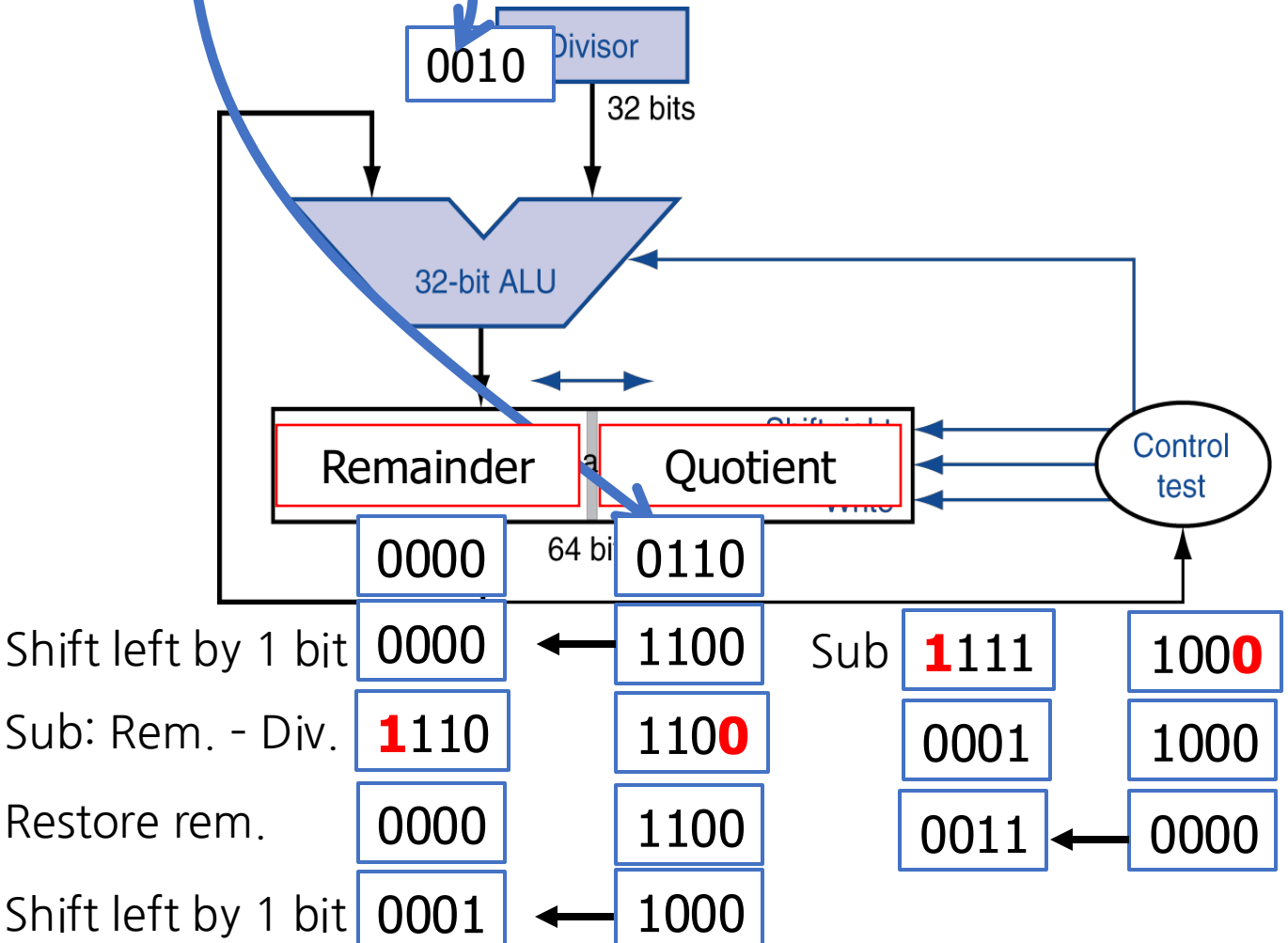
- $6_{10} / 2_{10} = 0110_2 / 0010_2$



	0000	0110		
Shift left by 1 bit	0000	1100	Sub 1111	1000
Sub: Rem. - Div.	1110	1100		
Restore rem.	0000	1100		
Shift left by 1 bit	0001	1000		

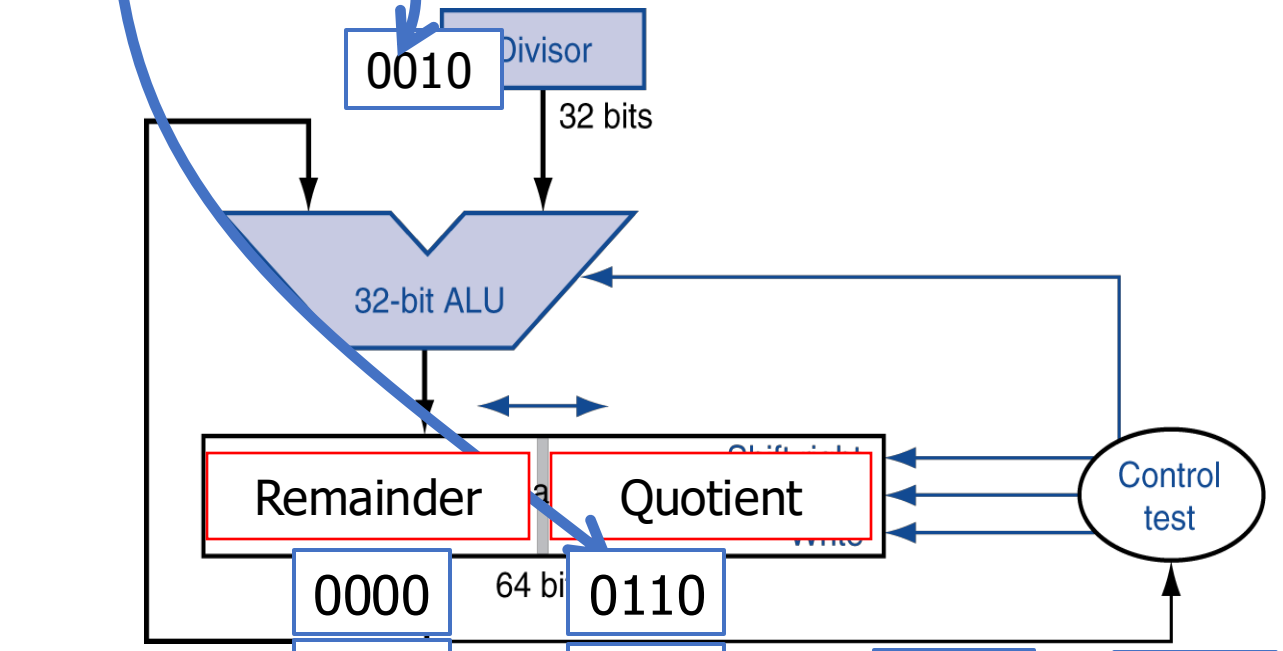
Division Example

- $6_{10} / 2_{10} = 0110_2 / 0010_2$



Division Example

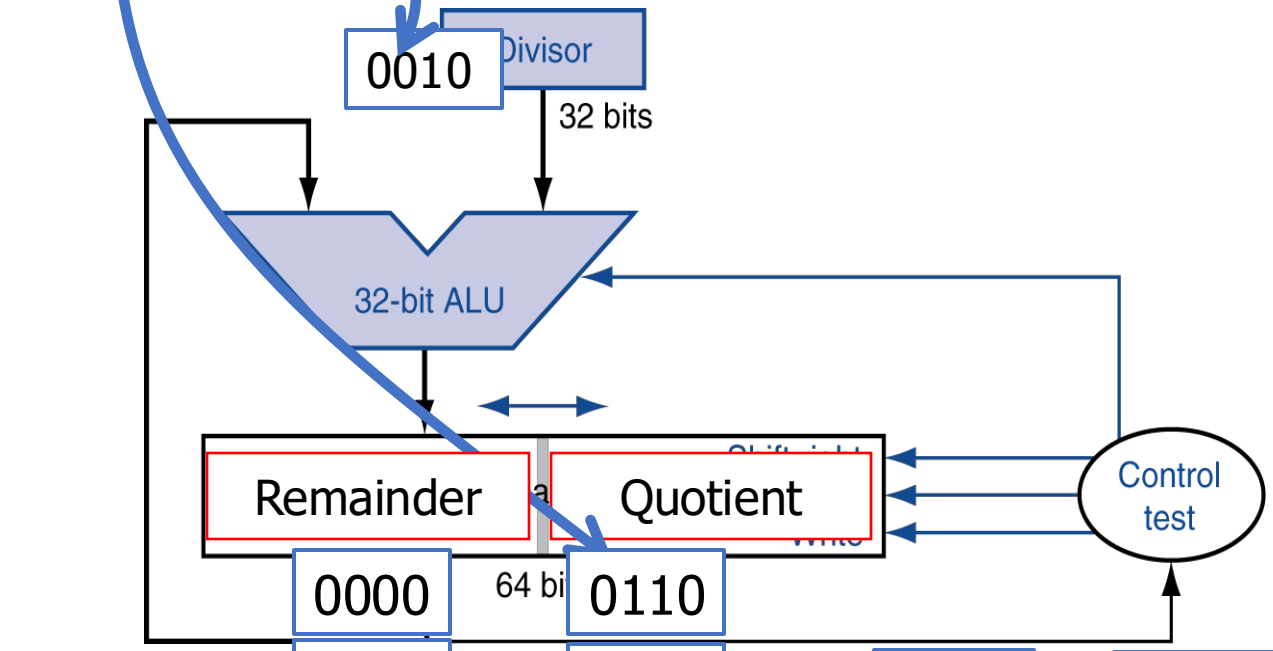
- $6_{10} / 2_{10} = 0110_2 / 0010_2$



	0000		0110			
Shift left by 1 bit	0000	←	1100	Sub	1111	1000
Sub: Rem. - Div.	1110		1100		0001	1000
Restore rem.	0000		1100		0011	← 0000
Shift left by 1 bit	0001	←	1000	Sub	0001	0001

Division Example

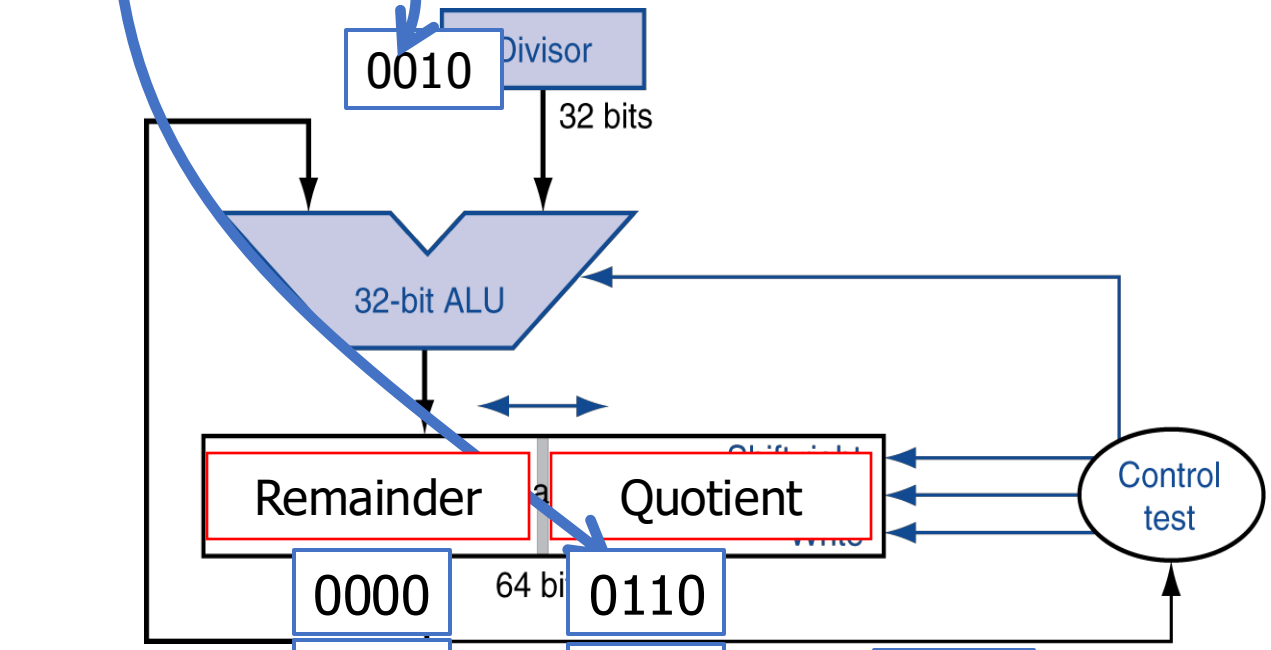
- $6_{10} / 2_{10} = 0110_2 / 0010_2$



	0000	0110					
Shift left by 1 bit	0000	1100	Sub	1111	1000	0010	0010
Sub: Rem. - Div.	1110	1100		0001	1000		
Restore rem.	0000	1100		0011	0000		
Shift left by 1 bit	0001	1000	Sub	0001	0001		

Division Example

- $6_{10} / 2_{10} = 0110_2 / 0010_2$



	0000		0110					
Shift left by 1 bit	0000	←	1100	Sub	1111	1000	0010	← 0010
Sub: Rem. - Div.	1110		1100		0001	1000	0000	0011
Restore rem.	0000		1100		0011	← 0000		
Shift left by 1 bit	0001	←	1000	Sub	0001	0001		

More on Division

- Signed division
  - Divide using absolute values
  - Adjust sign of quotient and remainder as required
    - 7 / 2: Q = 3, R = 1
    - (-7) / 2: Q = -3, R = -1
    - 7 / (-2): Q = -3, R = 1
    - (-7) / (-2): Q = 3, R = -1
- Faster division
  - Can't use parallel hardware as in multiplier
    - Subtraction is conditional on sign of remainder
  - Faster dividers (e.g. SRT division) generate multiple quotient bits per step, but still require multiple steps

RISC-V Division

- Four instructions:
  - Signed

```
div  rd, rs1, rs2 # rd = rs1 / rs2
rem  rd, rs1, rs2 # rd = rs1 % rs2
```

  - Unsigned

```
divu rd, rs1, rs2 # rd = rs1 / rs2
remu rd, rs1, rs2 # rd = rs1 % rs2
```

- Overflow and division-by-zero don't produce errors
  - Just return defined results
  - Faster for the common case of no error